

Research

Outpatient scheduling problem in smart hospital with two-agent deep reinforcement learning algorithm

Jianing Zuo¹ · Yan Jin² · Weibo Liu¹

Received: 14 June 2024 / Accepted: 11 October 2024

Published online: 06 November 2024

© The Author(s) 2024 **OPEN**

Abstract

As the internet and digital technologies revolutionize the healthcare sector, the development of smart hospitals is pivotal for improving medical service efficiency. This research presents a mathematical model for the outpatient scheduling problem and introduces the Graph-based Two-Agent Deep Reinforcement Learning algorithm (GTARS-DRL) to address the escalating demand for medical services and the scarcity of resources. The GTARS-DRL leverages graph neural networks for feature extraction of patient medical activities and applies the PDQPN algorithm to train agents for optimizing patient sequencing and service desk allocation, effectively minimizing wait times. Our experiments indicate that GTARS-DRL outperforms traditional scheduling rules in solution quality and matches the performance of Genetic Algorithms (GA), with a notable advantage in computational efficiency, boasting an ACPU of 0.762 s and an ARPT of 0.327 s. The algorithm's efficacy is further supported by its successful application to larger-scale instances, showcasing its potential for real-world use. This study contributes a novel approach to outpatient scheduling in smart hospitals, providing theoretical backing and practical direction for the optimization of medical resource allocation, and is poised to significantly enhance service efficiency and patient satisfaction.

Keywords Outpatient scheduling · Graph Neural Networks · Markov Decision Process · Two-agent · Proximal Policy Optimization

1 Introduction

With the widespread application of the internet and digital technology, the healthcare sector is undergoing revolutionary changes. As emerging technologies like artificial intelligence, precision medicine, and telemedicine become increasingly incorporated into the healthcare sector, there is a concurrent transformation in the expectations patients hold for medical services [1, 2]. More and more patients expect to receive efficient and convenient medical services in their daily lives. In this context, hospitals, as the core of the healthcare service system, need to undergo self-reform through intelligent upgrades to establish smart hospitals. The ongoing refinement of research in the domain of scheduling problems is propelling their implementation within the healthcare industry, thereby accelerating the evolution of smart hospitals [3].

With the continuous increase in people's medical needs, limited medical resources are gradually unable to meet the patient's treatment needs promptly. In the United States, a shortage of 1.1 million nurses by 2022 looms large alongside increasing demand for medical professionals due to the aging population. Japan faces similar challenges, expecting a shortfall of nearly one million medical personnel by 2040. China, following healthcare reforms since 2009,

✉ Weibo Liu, liuweibo@sdu.edu.cn | ¹School of Management, Shandong University, Ji'nan, Shandong, China. ²School of Mechanical and Aerospace Engineering, Queen's University Belfast, Belfast, UK.



witnessed 8.42 billion medical consultations in 2022. Rising medical demand globally strains limited resources, leading to prolonged patient wait times and inefficiencies in care delivery. At the same time, hospitals lack scientific and effective control over the diagnosis and treatment processes of patients, resulting in patients spending a significant amount of time waiting during their medical visits.

Faced with the growing demand for medical treatment, hospitals cannot simply rely on purchasing key medical equipment or increasing medical staff to meet the patients' diagnosis and treatment needs. Optimizing intelligent patient scheduling to improve the efficiency of medical services is also a prominent issue that smart hospitals face in creating a scientific healthcare service management system. Although China has introduced measures like online appointment booking, addressing long wait times during consultations, examinations, and follow-up visits remains crucial. Current optimization strategies have predominantly focused on enhancing specific elements of hospital operations, including scheduling for outpatient appointments, organizing surgical theater timetables, and distributing diagnostic resources [4–6]. However, these efforts have not yet coalesced into an integrated framework aimed at curtailing the cumulative duration of patients' engagement in various medical procedures within the hospital setting. There remains a significant opportunity to develop a comprehensive approach that addresses the totality of patient time investment across all hospital-based medical activities, thereby enhancing operational efficiency and patient care experience. This underscores the significance, complexity, and urgency of improving medical service efficiency through effective patient scheduling.

The patient scheduling problem is akin to workshop scheduling. Workshop scheduling optimizes production efficiency by strategically allocating resources and tasks to achieve peak performance, crucial in industries like manufacturing where efficiency is paramount. According to the workshop scheduling problem, the available resources are commonly referred to as 'machines', and the tasks awaiting processing are called 'jobs'. In medical scheduling, machines can be regarded as medical activities, and jobs can be seen as patients in need of medical activities.

While workshop scheduling typically involves a fixed sequence of operations tailored to specific machines, the complexity of hospital patient scheduling lies in its inherent flexibility. Unlike the structured workshop environment, where job processes are predefined and mapped directly onto designated machinery, patient medical activities in hospitals do not adhere to a set order. This fluidity in patient scheduling introduces a level of complexity akin to that of flexible job shop scheduling problems, where the priority and sequence of tasks can vary, presenting a more intricate challenge that must be navigated with sophisticated scheduling strategies [7]. The variability in the order of medical activities for patients necessitates a more adaptable approach, one that can dynamically respond to the individualized needs and priorities of each case, thus elevating the scheduling problem in hospitals to a higher plane of complexity. For instance, when a patient is diagnosed with COVID-19, they may undergo a series of diagnostic procedures such as blood tests, electrocardiograms, and computed tomography scans, which can be performed in any sequence rather than following a set, rigid order. Additionally, most medical activities involve multiple service desks. Therefore, in the process of outpatient patient scheduling, besides considering the ordering of patients in each medical activity, it is also necessary to determine the sequence of medical activities for each patient and select the service desk for each medical activity. These factors contribute to the increased complexity of outpatient patient scheduling.

Outpatient patient scheduling aims to minimize the total time spent by all patients receiving treatment. The time for each medical activity includes waiting time and service time, where waiting time refers to the period when all service desks for a specific medical activity are occupied upon a patient's arrival, requiring the patient to queue until they can start receiving the medical service, and service time refers to the actual time spent by a patient receiving medical treatment. Since the service time for each medical activity is generally constant, the essence of patient scheduling is essentially to minimize the waiting time for each patient.

In our algorithmic design for patient scheduling in medical activities, we have made the initial assumption regarding the specific medical activities patients will engage in. This approach is justified by two key considerations. Firstly, with the advent of predictive diagnostics in advanced healthcare settings, intelligent systems are now capable of recommending a set of medical procedures based on a patient's pathophysiological symptoms. This proactive method is indicative of a future where patients can be informed of necessary medical activities without the need for an initial consultation, thereby simplifying the current complex healthcare process. Secondly, it is observed that certain outpatient procedures, such as those involved in cancer treatment, follow a relatively fixed routine.

This research introduces for the first time a dual-agent methodology within the context of hospital patient scheduling, culminating in the innovation of the Graph-based Two-Agent Deep Reinforcement Learning algorithm (GTARS-DRL). By employing a Markov Decision Process and a multi-action space, this algorithm adeptly manages the intricacies of hospital scheduling, considering both the sequencing of patient medical activities and the strategic selection

of service desks. This holistic strategy is designed to streamline patient flow, markedly decrease waiting times, and augment the efficiency of resource utilization within hospitals. The study adeptly addresses a significant void in current scheduling systems regarding their capacity to manage substantial patient volumes and resource distribution, underpinned by empirical validation of the proposed approach's efficacy. Key contributions of this research to outpatient scheduling in smart hospitals include:

- i. The formulation of the outpatient scheduling challenge into a precise mathematical model, establishing a robust foundation for analytical rigor.
- ii. The introduction of a disjunctive graph representation for scheduling states, presenting an innovative paradigm for tackling scheduling complexities.
- iii. The development and implementation of the Proximal Deep Q-Policy Network (PDQPN) algorithm, which refines agent policies to accomplish highly efficient scheduling optimization.

An overview of this study is shown in Fig. 1. The subsequent sections of this paper are organized as follows: Section 2 provides a literature review, detailing the research on scheduling problems and the motivations behind the method proposed in this paper; Sect. 3 presents the mathematical model and disjunctive graph model for outpatient Scheduling problems; Sect. 4 introduces the design of a Markov Decision Process based on the GTARS-DRL method, along with the feature extraction method using Graph Isomorphism Network (GIN) and fully connected layers, followed by training using the PDQPN algorithm; Sect. 5 demonstrates the effectiveness and generality of the PDQPN algorithm through numerical experiments; and Sect. 6 presents the conclusions of this study.

2 Literature review

2.1 Outpatient scheduling

Outpatient departments act as the gateway for hospital services, offering patients the convenience of appointment registration through diverse platforms such as the internet, telephone, and SMS. The patient visit encompasses a spectrum of processes beyond mere registration, including payment, queuing, and consultation. However, large

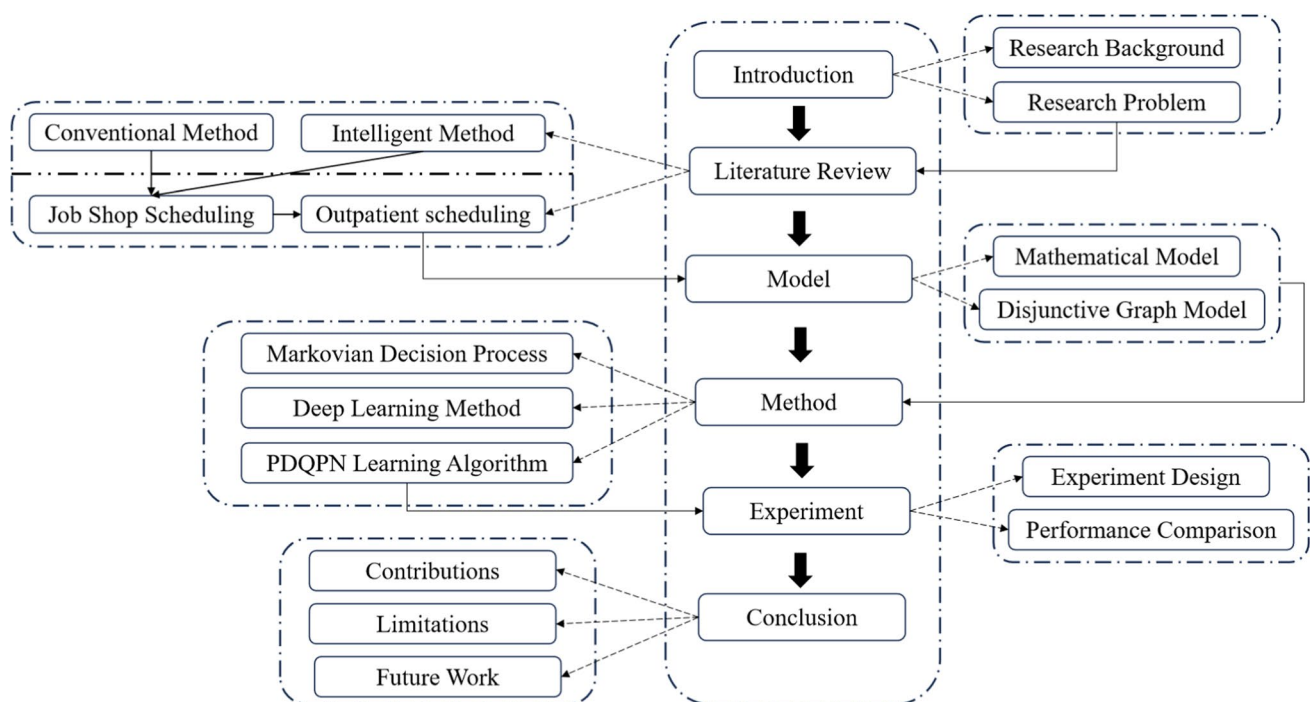


Fig. 1 Research overview

hospitals frequently encounter the “three longs and one short” dilemma—extended waiting times for registration, consultation, and payment contrasted with brief actual consultation periods. This can result in patients spending more time waiting than engaged in medical consultation, leading to a significant waste of time and a precipitous drop in satisfaction levels.

Existing outpatient scheduling systems have primarily focused on the allocation of medical staff and equipment, as evidenced by research into outpatient appointment systems that aim to streamline registration processes [8, 9], strategies for adjusting the number of service desks to accommodate varying patient volumes [10], and methods for scheduling nurses that consider specific patient conditions to enhance care delivery and reduce waiting times [11, 12]. While these efforts are critical, they fall short of addressing the root causes of inefficiency. Therefore, intelligent patient flow management and behavior control have emerged as pivotal to resolving these systemic issues.

Research in outpatient scheduling has predominantly focused on individual medical activities, an approach that falls short of addressing the extensive waiting times and overcrowded conditions commonly found in large hospitals. For instance, although progress has been made in optimizing scheduling within specific medical activities to reduce wait times—such as streamlining admission and recovery processes in outpatient operating rooms [13, 14]—these efforts do not adequately tackle the overarching systemic challenges. There is an urgent need for a comprehensive strategy that takes into account the patient’s journey in its entirety. Furthermore, while certain departments, including internal medicine, radiology, and pharmacies, have seen targeted efforts to enhance patient flow and improve scheduling efficiency [4–6], these advancements remain isolated and do not lend themselves to a cohesive solution applicable throughout the hospital. The compartmentalization of these improvements highlights the critical need for an integrated patient scheduling approach. Such an approach would transcend the limitations of individual activities and departmental silos, ensuring a seamless and efficient patient experience across the entire hospital facility.

This necessity for integration becomes even more evident when considering the lack of research that takes into account the entire outpatient system. The current body of research has yet to fully explore how to manage the sequence of medical activities from a patient’s arrival to their departure, including the selection of appropriate service desks for each activity. This holistic consideration is crucial, as it involves understanding the sequential constraints within medical processes such as registration, consultation, diagnostics, follow-up visits, and medication dispensing. Additionally, the variability in the number of service desks—where some phases like consultation are limited to a single desk, while others, like diagnostics, may involve multiple desks—adds another layer of complexity to the scheduling challenge. Addressing these factors is key to developing a scheduling system that can meet the dynamic demands of modern healthcare delivery.

The outpatient scheduling problem shares many similarities with the Job Shop Scheduling Problem (JSP). Firstly, both involve the allocation of resources, such as service desks or machines, and the sequencing of tasks for patient medical activities or job processing. Secondly, whether it is job scheduling in a workshop or patient scheduling in medical services, the goal is to achieve optimal efficiency. In the outpatient patient scheduling problem, arranging the sequence of various medical activities for multiple patients can be viewed as similar to the assignment of machines for different operations in a job shop scheduling model. Given the fundamental similarities between patient scheduling and workshop scheduling, and considering the relatively mature research in workshop scheduling, this study systematically arranges all medical activities involved in patients’ outpatient visits based on methods used for workshop scheduling.

2.2 Metaheuristic approaches to job shop scheduling problem

Currently, methods for solving scheduling problems are primarily categorized into two approaches: exact solutions and approximate solutions. Exact solution methods, such as mathematical programming, identify the optimal solution within the entire solution space. However, due to their inherent limitations, these methods are only feasible for small-scale scheduling problems. Approximate solution methods encompass rule-based scheduling algorithms, metaheuristic algorithms, and machine learning algorithms. In outpatient scheduling, rule-based dispatching is divided into patient medical activity selection rules and service desk selection rules, which are combined to achieve the goal of minimizing the maximum completion time. Nevertheless, deriving a high-quality combination of scheduling rules often requires a substantial number of experiments for comparison, and relying on a single set of rules may not ensure the quality of the scheduling for the entire caseload.

Metaheuristic algorithms are the most commonly used optimization methods for solving scheduling problems. They include population-based and evolutionary algorithms such as Genetic Algorithms (GA), Particle Swarm Optimization (PSO), Wolf Pack Algorithm (WPA), and Grey Wolf Optimization (GWO). These algorithms search for local optima in scheduling problems through various iterative optimization operators. In recent years, scholars have extensively studied the Job Shop Scheduling Problem (JSP), with a focus on metaheuristic algorithms and traditional rule-based methods [15]. These algorithms utilize evolutionary operators or particle movements to iteratively search for optimal solutions [16]. Several metaheuristic algorithms have been applied to JSP, including Genetic Algorithm [17–19], Particle Swarm Optimization [20, 21], Ant Colony Optimization [22], and others. These methods are known for their effectiveness in obtaining high-quality scheduling solutions.

To enhance the exploration capabilities of these algorithms, researchers have introduced local search strategies and encoding methods. For instance, the Grey Wolf Optimizer has been improved with a maximum rank rule-based encoding [23]. Hybrid approaches, such as the Genetic Particle Swarm Algorithm, have also been developed to improve convergence speed [20]. More specific scenarios, like the distributed heterogeneous no-wait flow shop scheduling problem, have seen outstanding performance with algorithms like the Discrete Artificial Bee Colony Algorithm [24]. Novel approaches, such as the Hybrid Whale Optimization Algorithm, have been proposed to enhance search efficiency for global optimum solutions [25]. The use of the Improved Grey Wolf Optimizer for JSP has led to more effective solutions and computational efficiency through innovative encoding and decoding schemes and the introduction of new crossover operators [26].

Metaheuristics can achieve higher accuracy than rule-based methods and are widely applied across various scheduling problems. However, they have two main disadvantages. Firstly, the computational demand of these algorithms is high, and the inability to store parameterized models through pre-training means that each optimization must start from scratch, leading to long response times. Additionally, metaheuristic algorithms have poor generalizability; they often require different parameter tuning for different scheduling problems, making direct algorithm transfer challenging. In real-time scheduling, the extensive iterations of metaheuristic algorithms can result in excessive computation times, preventing the attainment of satisfactory outcomes. Despite the effectiveness of metaheuristic algorithms, the computational time required for solving large-scale scheduling problems remains a challenge. Traditional methods often require constant parameter adjustments, which can hinder adaptation to dynamic environments. In contrast, deep reinforcement learning, which combines deep learning with reinforcement learning, offers a promising approach to address these issues, providing superior solutions and computational efficiency [27–29].

2.3 Reinforcement learning in outpatient scheduling

With the advancement of artificial intelligence technologies, algorithms led by machine learning have achieved significant success in fields such as image recognition, natural language processing, and combinatorial optimization. Reinforcement learning, as a pivotal AI technique, offers the benefits of short response times and strong generalizability, particularly after training completion, making it suitable for addressing scheduling problems [30]. Moreover, extensive research applying reinforcement learning to scheduling issues has demonstrated its potential to achieve accuracy comparable to metaheuristic algorithms. The broad application of reinforcement learning in the field of scheduling presents a novel perspective for tackling patient scheduling challenges.

Reinforcement learning, which involves an agent learning to map environmental states to actions to maximize rewards, has introduced novel solutions to workshop scheduling [31]. Current research predominantly focuses on enhancing reinforcement learning algorithms and the design of Markov Decision Processes (MDP) for scheduling problems. Reinforcement learning algorithms are categorized into value-based and policy-based learning algorithms. Value-based algorithms, such as Q-learning and Deep Q-Network (DQN), evaluate expected returns using Q-tables or neural networks. Policy-based algorithms, including the Deep Deterministic Policy Gradient (DDPG) and proximal policy optimization (PPO), utilize neural networks for policy refinement through feedback signals.

However, to date, there has been a lack of research on how two-agent approaches can enhance the applicability of reinforcement learning algorithms to patient scheduling problems, which is the focal point of this study. This is because, within reinforcement learning algorithms, each agent is typically limited to executing a single action at each time step. In contrast, outpatient scheduling requires the simultaneous selection of patient medical activities and service desks. Therefore, in response to this challenge, this paper introduces a dual-agent reinforcement learning scheduling method tailored for outpatient scenarios. The proposed method leverages the strengths of two-agent systems to enable more flexible and efficient decision-making processes in patient scheduling. By allowing each agent to specialize in different

aspects of the scheduling task—such as patient medical activity selection and service desk allocation—the method aims to optimize the overall patient flow and resource utilization within the hospital setting. This innovative approach holds promise for improving the quality of care and operational efficiency in healthcare delivery.

Outpatient appointment scheduling presents unique challenges compared to workshop scheduling, with variations in the sequence of medical activities and the need for service desk selection. Given the complexity of outpatient scheduling, which involves continuous action sequences and high-dimensional state spaces, the PPO algorithm is selected for its stability and convergence benefits. PPO is adept at learning action probability distributions in various scheduling states, optimizing for maximum expected returns. This study employs PPO to train patient-medical activity agent, capitalizing on its suitability for environments with continuous action spaces and the need for robust decision-making in outpatient scheduling. In addressing the task of helping patients select the most suitable service desk, a discrete action decision-making problem involving choosing the best option from multiple candidates in medical activities—the Double Deep Q Network (DDQN) algorithm is utilized. DDQN, a value-based reinforcement learning technique combining Q-learning and DQN, alleviates overestimation problems and enhances Q-value estimation accuracy through two deep neural networks. Specifically tailored for scenarios requiring handling of discrete action spaces and value function estimation, DDQN aids patients in making optimal decisions during medical activities, ensuring the selection of the most appropriate service desk and minimizing overall time costs. Therefore, in this study, a Proximal Deep Q-Policy Network (PDQPN) algorithm is employed using policy-based PPO and value-based DDQN methods, depending on the specific objectives of each agent.

2.4 Reinforcement learning in job shop scheduling problem

Reinforcement learning methods in scheduling problems utilize real-time outputs of the environment as states, such as pending jobs and queueing times. The multi-AGV flow shop scheduling problem's optimization is addressed using Q-learning with full information sharing and Markov modeling [32]. An adaptive scheduling algorithm based on DQN, incorporating continuous-value range state features and heuristic scheduling rules, demonstrates superior performance in manufacturing compared to traditional methods and Q-learning algorithms [33]. To address overestimation in DQN, a DRL algorithm based on DDQN architecture was designed using Double Q-learning principles, aiming to minimize penalties for earliness and tardiness and establish a flexible scheduling system for self-learning and real-time optimization [34]. The PPO algorithm has been widely adopted to address optimization challenges in a discrete action space, especially in scenarios involving practical factors like processing time and material transfer in manufacturing processes [35, 36]. Additionally, the DDPG algorithm effectively trains a policy network using composite scheduling actions and continuous weight variables to achieve superior scheduling outcomes in workshop scheduling problems [37].

However, the aforementioned methods have the following drawbacks in terms of defining states. Adapting the algorithm to capture the differences between states for different scheduling problems requires significant adjustments. Additionally, these methods struggle to adequately represent the current scheduling environment, limiting their ability to extract hidden relationships within the scheduling environment effectively. To overcome these, integrating advanced models such as Graph Neural Networks (GNNs), pointer networks, and attention mechanisms can significantly improve the feature representation and adaptability of scheduling algorithms.

A disjunctive graph representation and GNNs have been utilized in an end-to-end Deep Reinforcement Learning (DRL) agent for automatic learning of Priority Dispatching Rules (PDRS) in Job Sequencing and Machine Selection Problems, showcasing robust performance on large-scale instances [38]. The Proximal Policy Optimization (PPO) algorithm has been employed for scalable learning in diverse scheduling tasks, outperforming classical heuristics with a single model [39, 40]. Recent advancements include the use of DQN and neural networks for approximating value functions in workshop scheduling [41], and the combination of DRL for PDRS learning with GNNs for heterogeneous graph scheduling state representation, enabling high-quality PDR learning [42]. Furthermore, the Graph Convolutional Network-based Two-Agent System (GMAS) has demonstrated exceptional performance in complex scenarios by constructing probabilistic models and adjusting scheduling strategies through decentralized interactions [43].

In summary, as encapsulated in Tables 1 and 2, while existing research has explored various scheduling problems, it has mainly focused on traditional job shop environments. This paper aims to investigate patient scheduling in hospitals. Most existing scheduling studies employ metaheuristic algorithms such as GA and PSO, which are prone to premature convergence and difficulties in parameter tuning, with limited extensibility in addressing the problem. Many reinforcement learning scheduling algorithms typically rely on a single-agent architecture, constrained by the requirements of neural network convergence, making it challenging to address problems involving complex scheduling actions. In the context of outpatient scheduling, considerations need to be given to patients, medical activities, and service desk selections. Existing methods

Table 1 Scheduling approaches in JSP and outpatient scheduling

Article	Job shop scheduling	Outpatient scheduling		Metaheuristic approaches	Reinforcement learning method	
		Single medical activity	Complete flow scheme		Value-based	Policy-based
Xin et al. [17]	√			√		
Chen et al. [18]	√			√		
Missaoui and Boujelbene [19]	√			√		
Liu et al. [20]	√			√		
Marichelvam et al. [21]	√			√		
Jieran et al. [23]	√			√		
Li et al. [24]	√			√		
Yang et al. [25]	√			√		
Li et al. [26]	√			√		
Xue et al. [32]	√				√	
Liu et al. [33]	√				√	
Chang et al. [34]	√				√	
Luo et al. [35]	√					√
Gil & Lee [36]	√					√
Gui et al. [37]	√					√
Park et al. [39]	√					√
Han & Yang [40]	√					√
Song et al. [42]	√					√
Jing et al. [43]	√					√
Golmohammadi et al. [8]		√				√
Kuiper and Mandjes [9]			√	√		
Wang et al. [13]		√		√		
Zhang et al. [14]		√		√		
Reihaneh et al. [4]		√		√		
Shao et al. [5]		√		√		
De Queiroz et al. [6]		√		√		
GTARS-DRL			√		√	√

limit each agent to executing only one action per time step, which fails to meet the demands of hospital patient scheduling. To address these issues, this paper proposes the GTARS-DRL for outpatient scheduling. It utilizes a hypergraph to represent the states of various medical activities for patients, incorporates GNN for state feature extraction, and establishes patient-medical activity agent and service desk agent for action selection. The strategies of these two types of agents are updated and learned through the PDQPN algorithm.

3 Problem statement

Current methodologies for solving patient scheduling problems are predominantly categorized into two distinct approaches. The first approach is characterized by exact solution methods, predominantly mathematical programming, which excel in translating scheduling issues into precise mathematical terms. However, these methods face limitations in resolving large-scale scheduling problems within a reasonable timeframe.

The second approach encompasses approximate methods, which include heuristics, metaheuristics, and machine learning techniques. With the advancement of artificial intelligence, reinforcement learning has demonstrated

Table 2 Advanced Scheduling Techniques Comparison

Article	State representation		High solution quality	High solving efficiency	Hidden relation representation	Agent	
	Ambient real-time output	Disjunctive graph				Single	Multi
Xin et al. [17]			✓				
Chen et al. [18]			✓				
Missaoui and Boujelbene [19]			✓				
Liu et al. [20]			✓				
Marichelvam et al. [21]			✓				
Jieran et al. [23]			✓				
Li et al. [24]			✓				
Yang et al. [25]			✓	✓			
Li et al. [26]			✓	✓			
Xue et al. [32]	✓			✓		✓	
Liu et al. [33]	✓			✓		✓	
Chang et al. [34]	✓		✓	✓		✓	
Luo et al. [35]	✓			✓		✓	
Gil & Lee [36]	✓			✓		✓	
Gui et al. [37]	✓		✓	✓		✓	
Park et al. [39]		✓		✓	✓	✓	
Han & Yang [40]		✓		✓	✓	✓	
Song et al. [42]		✓	✓	✓	✓	✓	
Jing et al. [43]		✓	✓	✓	✓		✓
Golmohammadi et al. [8]	✓			✓	✓	✓	
Kuiper and Mandjes [9]			✓				
Wang et al. [13]			✓				
Zhang et al. [14]			✓	✓			
Reihaneh et al. [4]			✓	✓			
Shao et al. [5]			✓				
De Queiroz et al. [6]			✓				
GTARS-DRL		✓	✓	✓	✓		✓

promising results when applied to scheduling problems. This paper presents the Graph-based Deep Reinforcement Learning Two-Agent Scheduling (GTARS-DRL) framework, which is specifically designed to address the scheduling issues of outpatients.

3.1 Mathematical model

The problem for outpatient patient scheduling in this paper is described as follows: assume there are n outpatient patients visiting a hospital. The treatment activities include m different medical procedures, such as registration, consultation, various types of examinations, follow-up visits, and medication collection. Each medical procedure is served by multiple service desks. The arrival times of patients follow a random distribution. For each outpatient, the first two medical procedures, registration and consultation, are fixed. After the consultation, doctors prescribe different examinations based on the patient's condition. Each patient has their own sequence of medical activities $\{\sigma_1^j, \sigma_2^j, \dots, \sigma_s^j\}$, and selecting different service desks for each activity corresponds to different durations $p_{j,i,q}$ required to complete that particular medical activity. Table 3 provides a detailed description of the model's requirements for binary variables, continuous variables, constant terms, and indexing symbols.

The optimization objective is to minimize the sum of the medical activity times for all patients, subject to the following constraints and assumptions:

Table 3 Index and variables

Index	
j, k	Patient index: 1, 2...n
i, h	Medical activity index: 1, 2...m
t	Index for different time instances
Binary variables	
$x_{k,j,i,q}$	Patient j followed by patient k is placed in line q at the service desk of medical activity i
$y_{j,i}$	Whether patient j needs to perform medical activity i
$a_{j,i,q,t}$	Whether patient j arrived at the q desk of medical activity i at time t
Continuous variable	
$C_{j,i}$	The time that patient j completes medical activities i until the end
$p_{j,i,q}$	The total time required for patient j to complete medical activities i after selecting service desk q
R_j	The time patient j arrived at the hospital
Constant	
N	Very large positive number

1. Each medical activity can only be carried out at one service desk at a time, with each desk serving only one patient. Within the same queue, each patient must wait for the preceding patient to complete their medical activity before commencing their own.
2. Each patient's medical activities are exclusive to a single service desk.
3. Each patient can undergo only one medical activity at a time, and subsequent activities must follow the completion of the previous one.
4. Once a patient's medical activity has commenced, it cannot be interrupted or halted.
5. The time spent by patients moving between two medical activities is disregarded.

Based on the description, definitions, constraints, and assumptions provided for the above problem, the mathematical model is established as follows:

$$\min \sum_j \left(\max_i C_{j,i} - R_j \right) \quad (1)$$

$$\sum_q \sum_{k=0, k \neq j}^n x_{k,j,i,q} = y_{j,i} \quad \forall_{j,i} \quad (2)$$

$$\sum_q \sum_{k=1, k \neq j}^n (x_{k,j,i,q} + x_{j,k,i,q}) \leq 2 \cdot y_{j,i} \quad \forall_{j,i} \quad (3)$$

$$\sum_q (x_{k,j,i,q} + x_{j,k,i,q}) \leq y_{j,i} \forall_{j,i,k} \quad (4)$$

$$\sum_q \sum_{j=1}^n x_{0,j,i,q} = 1 \quad \forall_i \quad (5)$$

$$\sum_t \sum_q a_{j,i,q,t} = y_{j,i} \forall_{j,i,q} \quad (6)$$

$$\sum_{i=1}^m \sum_q a_{j,i,q,t} \leq 1 \forall_{j,t} \quad (7)$$

$$C_{j,1} - R_j - \sum_t \sum_q a_{j,1,q,t} p_{j,1,q} \geq 0 \forall_{j,q} \quad (8)$$

$$\sum_q \sum_t (t + p_{j,\sigma_{h-1}^j,q}) a_{j,\sigma_{h-1}^j,q,t} \leq \sum_q \sum_t t a_{j,\sigma_h^j,q,t} \forall_{j,h} \quad (9)$$

$$\sum_q \sum_t (t + p_{j,i,q}) a_{j,i,q,t} \leq C_{j,i} \quad (10)$$

$$C_{j,i} \geq C_{k,i} + p_{j,i,q} + (x_{k,j,i,q} - 1) \cdot N \forall_{k \in \{1, \dots, n\} | j,i,q,j \neq k} \quad (12)$$

$$C_{j,i} \geq 0 \quad (11)$$

Equation (1) represents the objective function, which aims to minimize the total time taken for all patients to complete all medical activities. Equation (2) indicates that patient j is positioned behind a unique patient in the queue at service desk q for medical activity i . Equation (3) ensures that patient j has a uniquely determined position in the queue at service desk q for each medical activity i . Equation (4) prevents the occurrence of conflicting priorities, meaning that a patient cannot be both ahead of and behind another patient simultaneously. Equation (5) controls the virtual patient 0 to have exactly one patient j positioned behind it in each medical activity. Equation (6) signifies that each patient only needs to proceed to the area of medical activity i once. Equation (7) denotes that at any given time, a patient can appear in only one medical activity. Equation (8) ensures that patients can only undergo medical activities upon arrival at the hospital. Equation (9) enforces that the medical activity i for each patient cannot commence before the completion of the preceding medical activity. Equation (10) delineates the fundamental relationship between $C_{j,i}$ and $p_{j,i,q}$. Equation (11) states that if patient j is positioned after patient k in medical activity i , then the time for patient j to undergo medical activity cannot precede the time for patient k to complete medical activity i . Equation (12) guarantees that the completion time for each medical activity is non-negative.

3.2 Disjunctive graph model

A disjunctive graph model can provide a more intuitive depiction of the structure and relationships in scheduling problems, facilitating the transformation of scheduling problems into the shortest-path problems for resolution and thereby reducing computational complexity. The representation of the hospital patient scheduling problem using a disjunctive graph requires an initial encoding of the problem with an undirected graph $G = (O; C_u \cup D_u)$, where O represents the set of all vertices in the directed graph, with each vertex denoting different medical activities for various patients; C_u denotes the set of undirected edges connecting adjacent medical activities for the same patient in graph G ; and D_u represents the set of undirected edges connecting the same medical activity for different patients in graph G .

The essential decision in patient scheduling involves determining the sequencing of different patients for the same medical activity and the sequencing of different medical activities for the same patient. This process involves transforming the undirected edges in Fig. 1a into directed edges in Fig. 1b. Once the order of medical examinations for each patient is established, C_u transitions into directed arcs C_d , with the direction of the arcs indicating the order of adjacent medical activities for each patient. When the order of all patients corresponding to each medical activity is completely determined, D_u transforms into directed arcs D_d , where the direction of the arcs indicates the order in which the two connected patients undergo the medical activity. The union of C_d and D_d is referred to as the disjunctive arcs of the disjunctive graph.

A necessary condition for a patient to undergo each medical activity is that the preceding activity associated with the corresponding vertex on the disjunctive graph has been completed. The collection of directed edges S in Fig. 1b

is termed a solution. A solution S is considered a feasible schedule only if it satisfies the following conditions: (1) all disjunctive arcs are fixed, and (2) the graph $G(S) = (V, S)$ does not contain any cycles.

A patient scheduling analysis model is established using coronary heart disease (CHD) examinations as an example. In the diagnosis of CHD, three examinations are typically required: complete blood count (CBC), electrocardiogram (ECG), and computed tomography (CT). The sequence in which the patient undergoes these three examinations is arbitrary, but at any given time, only one examination can be performed (Table 4).

According to the above existing scheduling information, the disjunctive diagram can be shown in Fig. 2.

In a feasible schedule, the completion time of a medical activity for each patient is when the patient finishes the medical treatment. Figure 3 depicts the Gantt chart corresponding to the directed acyclic graph with finalized directions in Fig. 1.

4 New GTARS-DRL method

This section introduces the principle of a graph-based deep reinforcement learning two-agent scheduling (GTARS-DRL) method. The GTARS-DRL method first defines outpatient scheduling as a Markov Decision Process (MDP). Different from previous outpatient scheduling methods, the GTARS-DRL method represents patient medical activities through a disjunctive graph and extracts the state features related to patient-medical activity and service desk using graph neural

Table 4 The time during which the patient performs various medical activities

	Time(min)		
	Patient 1	Patient 2	Patient 3
CT	2	4	7
CBC	7	4	4
ECG	2	1	6

Fig. 2 Scheduling disjunctive graph model

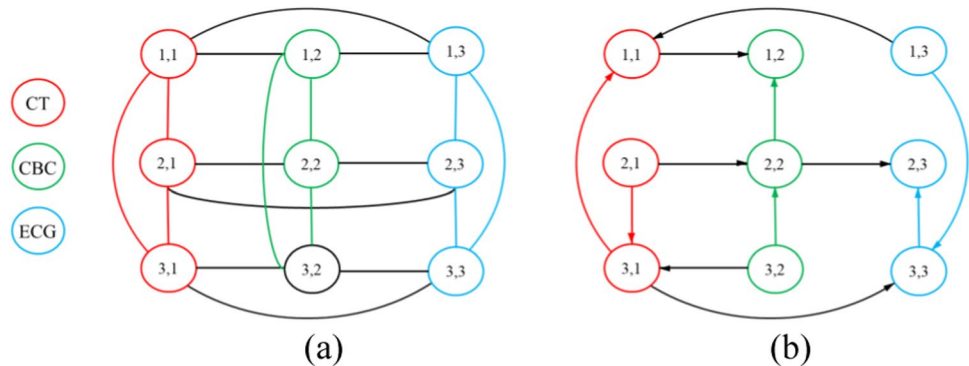
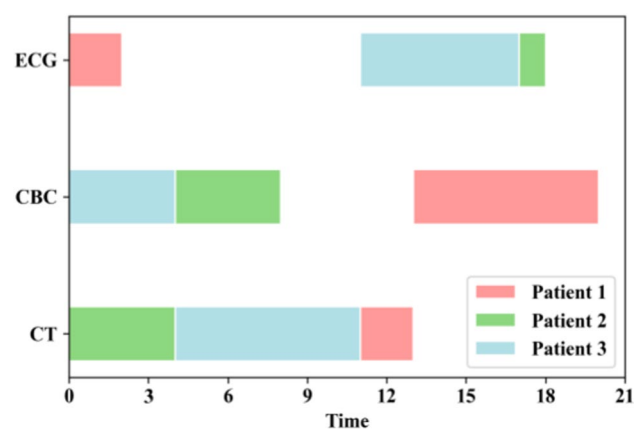


Fig. 3 Patient scheduling Gantt chart



networks and fully connected layers. Therefore, the GTARS-DRL method effectively captures the features of the scheduling environment and the complex relationships between scheduling entities.

The GTARS-DRL framework is illustrated in Fig. 4. GTARS-DRL uses the patient-medical activity agent to determine the sequence of medical activities for each patient and the order of patients in each medical activity. The service desk agent assists patients in selecting the shortest wait time for different medical activities. GTARS-DRL can be trained on a small-scale patient scheduling dataset and scaled to large-scale patient scheduling. In addition, this paper designs a Proximal Deep Q-Policy Network (PDQPN) training algorithm to effectively train multiple agents in GTARS-DRL. PDQPN fully leverages the advantages of value-based and policy-based reinforcement learning methods, addressing scalability and environmental variability challenges in patient scheduling within hospitals.

4.1 Markovian decision process (MDP)

Common scheduling problems can be modeled as a type of MDP, where the Markov property requires that in a random process, the future state depends only on the current state and is independent of past states. In outpatient scheduling problems, when arranging patients for medical procedures, only the current patient—medical activity status and the status of each service desk for medical activities need to be considered to determine the next action to take, without taking into account the previous situations of patients, medical activities, and service desks.

Outpatient scheduling problems involve arranging the order of different patients for various medical procedures and considering how to schedule patient treatment when there are multiple service desks for the same procedure. This involves coordinating two agents to control multiple actions simultaneously. This approach based on GTARS-DRL for problem-solving can be modeled using an MDP tuple $\{S, A, P, R\}$: the state space S contains the set of environmental states s ; the action space A includes all available actions a ; P is the state transition probability function, where $P(s'|s, a)$ indicates the probability of transitioning from state s to state s' by taking action a ; R represents the reward function, with reward r denoting the reward obtained by performing a certain action in a particular state. The primary objective of deep reinforcement learning is to learn from a series of action decisions, and take actions that lead the agent to maximize cumulative rewards in the future.

This study formulates the outpatient scheduling problem as a MDP, a model capable of predicting future states based on the current state, thereby providing decision support to reduce patient wait times and enhance scheduling efficiency. The core advantage of MDP lies in its ability to simulate the transitions of patients between different

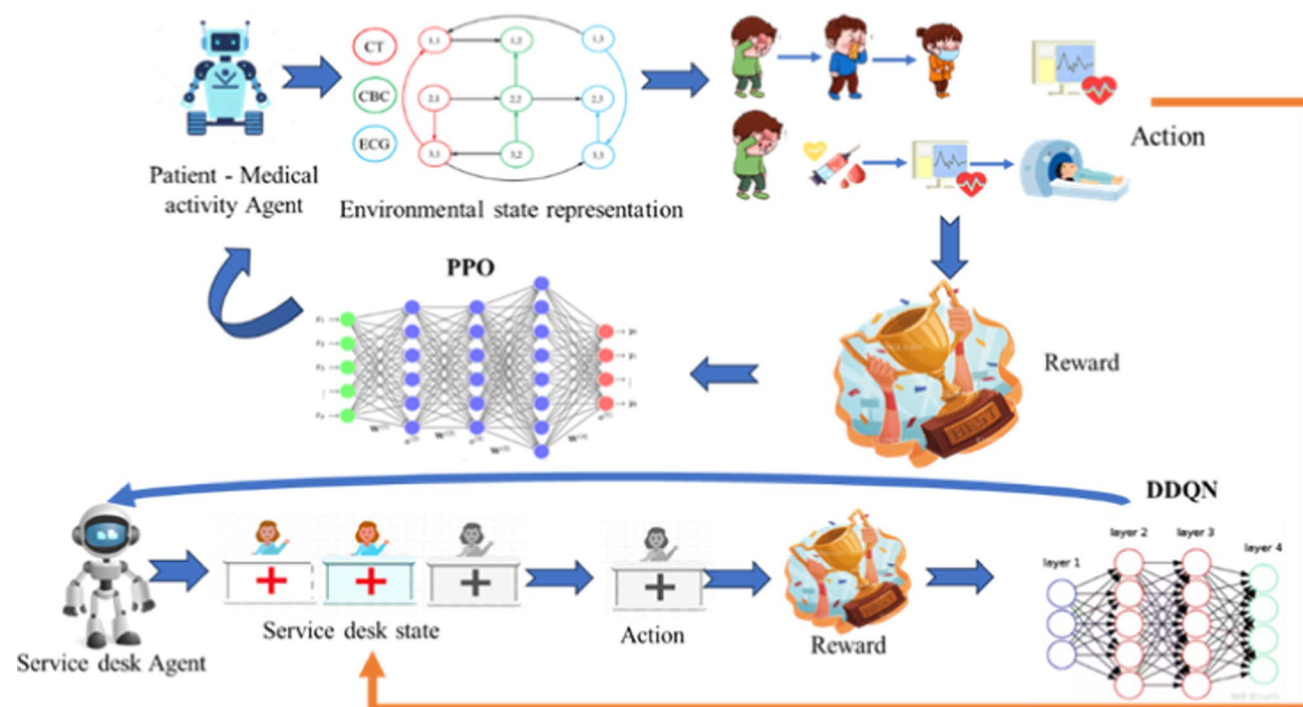


Fig. 4 The framework of the GTARS-DRL

medical activities and how these transitions impact overall scheduling efficiency. By defining the state space S , action space A , transition probability P , and reward function R , the model in this research focuses on learning how to select the optimal action in the current state to maximize cumulative rewards, which is directly related to the goal of reducing the total waiting time for patients.

Definition 1 state In the Markov model of patient scheduling, the state serves as a crucial indicator reflecting patient scheduling information. To conduct comprehensive and accurate analyses, it is essential to represent the current state information in a refined manner so that intelligent agents can be effectively trained. Moreover, appropriate state representation enables generalization for application, allowing well-trained neural networks to be applied to unseen and varying-scale problems.

During the patient scheduling process, both patients and medical activities possess individual states. For instance, parameters such as the number of patients in the hospital, remaining patient count, utilization rates of each medical activity, or waiting queue lengths can reflect the current state of the hospital environment and serve as the basis for subsequent scheduling decisions. Most reinforcement learning-based scheduling methods utilize these states or computed state features for decision-making. However, some features have inherent limitations. For example, the range of values for the remaining patient count may vary significantly, making it challenging for models to accurately assess the current state when using such features for learning, thereby reducing model performance and adaptability. Therefore, to overcome these limitations, this study adopts a disjunctive graph representation of the environmental state, achieving improved scheduling quality and generalization performance.

At decision point t , the global state s_t consists of local states s_{ot} and s_{mt} . As per the content in Sect. 3.2, the state s_{ot} is represented by a disjunctive graph $G(t) = (O; C_{dot} \cup D_{uot}, C_{ut} \cup D_{ut})$, where C_{dot} and D_{uot} denote the set of disjunctive arcs with predetermined directions before decision point t , and C_{ut} and D_{ut} are the collection of remaining disjunctive arcs. The basic scheduling information of hospital patients is transformed into a disjunctive graph, where each patient's involvement in various medical activities corresponds to nodes in the graph, and the sequence of medical activities corresponds to arcs in the graph. Each node's information includes whether the patient has completed the medical activity and the total time spent participating in that activity. For each node $O_{ji} \in O$, this study records two features $I(O_{ji}; s_t), CLB(O_{ji}; s_t)$ [44]:

1. The binary indicator $I(O_{ji}; s_t)$ equals 1 only when activity O is scheduled in state s_t .
2. The integer $CLB(O_{ji}; s_t)$ represents the estimated lower bound on the completion time of medical activity O in states s_t . For activities already scheduled, this lower bound corresponds to their completion times. For medical activity O_{ji} not yet scheduled for patient j , this lower bound can be recursively calculated as $CLB(O_{ji}; s_t) = CLB(O_{j,i-1}; s_t) + p_{j,i,q}$. If O_{j1} is the first medical activity for patient j , then $CLB(O_{j1}; s_t) = r_j + p_{j,1,q}$, where r_j is the arrival time of patient j .

The other local states s_{mt} , comprises the states of all service desks, where each state includes two-dimensional features $[H_t(M_{iq}), p_{j,i,q}]$:

1. $H_t(M_{iq})$ is the task completion time of the service desk q of medical activity i before decision point t .
2. $p_{j,i,q}$ denotes the sum of waiting and examination times for patient j at service desk q of medical activity i .

Definition 2 action The scheduling actions involve the simultaneous determination of the selection of service desks for each medical activity, the order of different medical activities for each patient, the order of patients within the same medical activity, and the start times of medical activities for patients. Therefore, the action space in this study is dual-layered, comprising the patient-medical activity selection action space and the service desk selection action space. The action set $A = A_o \times A_m$ is two-dimensional, corresponding to the selection of medical activities and patients by the patient-medical activity agent and the selection of service desks by the service desk selection agent. Here, A_o is a subset of candidate medical activity actions for eligible patients, and A_m is a subset of actions compatible with multiple service desks. At time t , the action a_t consists of two actions, $a_{ot} \in A_{ot}$ and $a_{mt} \in A_{mt}$, where a_{ot} represents the selection of medical activities and patients, and a_{mt} represents the selection of service desks for medical activities. The action a_t is defined as (O_{ji}, m_q) , indicating the assignment of a specific medical activity i to a patient j and the allocation of an appropriate service desk q .

Definition 3 state transition At scheduling time t , the completion of a medical activity by a patient triggers the progression to the next time step $t + 1$. Following each scheduling completion, the environment transitions from the current state s_t and the action chosen by the agent, a_t , to a new state s_{t+1} . Specifically, in the disjunctive graph, the node and arc information is updated based on the selected patient and medical activity, with arcs being modified in the directed graph. Simultaneously, the completion time of the medical activities for the corresponding patients is updated to generate a new disjunctive graph as the states s_{t+1} . Subsequently, the end time of the medical activity at the corresponding service desk, $H_{t+1}(M_{iq})$, and the processing time p_{jiq} associated with a_{0t} are updated to form the new local states $s_{m(t+1)}$.

Definition 4 reward In the hospital patient scheduling problem, the objective is to gradually learn how to schedule in order to minimize the total time required for all patients' treatments. To achieve this goal, we need to design a corresponding reward function. The difference in total time taken by all patients for medical activities between two consecutive state transitions in the directed graph of the corresponding scheme, denoted as $r(t) = T(s_t) - T(s_{t+1})$, serves as the immediate reward for decision-making. Here, $(T(\cdot))$ represents a quality metric defined as the total time required for all patients' treatment, calculated by:

$$T(s_t) = \sum_j \left(\max_i CLB(O_{j,i}; s_t) + p_{j,i,q} \right) \quad (13)$$

The final computed total reward value is given by:

$$R = \sum_{t=1}^{end} r(t) = T(s_0) - T(s_{end}) \quad (14)$$

Here, $T(s_0)$ refers to the sum of completion times for all patients' medical activities in the initial state, which is a fixed value, and $T(s_{end})$ corresponds to the completion time after the agent executes the last action. It can be observed that maximizing the reward value R aligns with the objective of minimizing the total treatment time for all patients.

Modeling the outpatient scheduling problem as an MDP provides a solid theoretical foundation for intelligent decision-making. This study will then explore how to enrich this model using deep learning methods to further enhance our scheduling strategies. Deep learning techniques, particularly Graph Neural Networks (GNNs), will be introduced in Sect. 4.2 to demonstrate how they capture the complexity and contextual information of patient medical activities, thereby supporting the goals of this research to reduce patient waiting times and improve scheduling efficiency.

4.2 Deep learning method

To effectively address the complexity and high dimensionality inherent in outpatient scheduling problems, the GTARS-DRL method employs deep learning approaches to augment the MDP model. This section will detail how GTARS-DRL utilizes a disjunctive graph to comprehensively capture key contextual information in the hospital patient scheduling process, such as the sequence of patients' various medical activities and constraints on the priority order of patients within medical activities. Specifically, the GTARS-DRL method utilizes Directed Acyclic Graphs to capture contextual information in hospital outpatient scheduling, including the sequence of patient activities and priority constraints. Through the aggregation and transformation capabilities of GNNs, the GTARS-DRL method is able to more effectively extract features related to patients, activities, and front-desk services, and capture the complex relationships between patients and medical activities. The Graph Isomorphism Network (GIN) is particularly well-suited for this task, as it updates based on node features and information from neighboring nodes, thus providing inputs for the deep reinforcement learning model. These features assist the intelligent agents in better understanding the current state, leading to decisions that can reduce patient waiting times. Additionally, this study designs a fully connected layer encoder for the service desk status to handle state information that lacks graph structure, allowing intelligent agents to comprehensively consider the current state of the service desk and the needs of the patients.

The GTARS-DRL method aims to identify environmental states and generate behavioral policies for each agent based on these states. The encoders are responsible for parsing the structural information provided by the disjunctive graph and transforming it into state representations that reflect key attributes of patients, medical activities, and service desks. The decoders then generate corresponding action decisions based on these state representations, namely patient-medical activity selection policies and service desk selection policies. By employing this deep learning

strategy, structured information in hospital patient scheduling is effectively transformed into the decision-making process of agents, thereby achieving intelligent optimization of the scheduling process [44].

Definition 5 patient-medical activity encoder The GNN is a model capable of handling graph-structured data by conducting information propagation and aggregation on nodes and edges, enabling representation learning for the entire graph, and it is utilized in reinforcement learning tasks. The GIN is a variant of GNNs that updates based on node features and information from neighboring nodes, further enhancing its ability to model graph structures. It is frequently applied to tasks such as node classification and link prediction. In the process of solving the disjunctive graph in patient scheduling at hospitals, the primary objective is to select nodes and determine the direction of disjunctive arcs. This aligns well with the functionality of the GIN model.

Therefore, this study adopts the GIN model as the feature extraction model for the local state s_{ot} . The local state s_{ot} , as input to the deep reinforcement learning model, calculates the features of the nodes in the graph using GIN and then feeds them into a linear decision model to obtain the probability distribution for patient-medical activity selection. Based on the resulting probability distribution, the medical activity for the patient with the highest probability is selected. By employing the GIN graph neural network for state extraction in the disjunctive graph, it is more conducive for the agent to identify the current state, thereby producing better action outputs.

The formula for updating node information refers to the equation used in GIN to calculate the feature values of individual nodes, which collectively generate the embedding representation of the entire graph. In the disjunctive graph, each node undergoes encoding through K layers of GIN networks, where each layer employs a Multi-Layer Perceptron (MLP) for computation. Within each layer, the node updates adhere to the following expression:

$$\mathbf{h}_{v,t}^{(k)} = \text{MLP}_{\theta_k} \left((1 + \epsilon^{(k)}) \mathbf{h}_{v,t}^{(k-1)} + \sum_{u \in N(v)} \mathbf{h}_{u,t}^{(k-1)} \right) \quad (15)$$

In Eq. (15) v denotes a node, $\mathbf{h}_v^{(k)}$ represents the feature of node v in the k -th layer, θ_k stands for the parameters of the MLP in the k -th layer, and $1 + \epsilon^{(k)}$ signifies the weight of the previous feature when aggregating the next feature. Additionally, $\sum_{u \in N(v)} \mathbf{h}_u^{(k-1)}$ corresponds to the weighted sum of the features of other nodes adjacent to v .

In the original GIN model, it was initially designed to handle undirected graphs. However, in this study, the disjunctive graph $G(t)$ associated with each state s_{ot} becomes a mixed graph with directed arcs after determining the order of various medical activities, capturing crucial features such as priority constraints and patient sequencing for each medical activity. Therefore, an extension of GIN is needed to support the processing of directed graphs. To achieve this goal, a natural and direct strategy is to replace each undirected arc in $G(t)$ with two directed arcs connecting the same nodes in opposite directions, resulting in a fully directed graph denoted as $G_D(t)$. The neighborhood of node (v) can be defined as $N(v) = \{(u, v) | (u, v) \in E(D(t))\}$, where $E(\cdot)$ represents the edge set of the graph. In other words, $N(v)$ contains all incoming neighbors of v . However, this strategy often fails to effectively extract disjunctive graph features using GIN due to the dense nature of the graph. Therefore, this study adopts an “addition of arcs” strategy by disregarding the initial undirected arcs to reduce computational complexity. Through these operations, $G_D(t)$ can be processed similar to the original GIN model in graph neural network computations. The encoder ultimately outputs node embeddings $\mathbf{h}_{v,t}^{(k)}$ and the entire graph pooling vector $\mathbf{h}_g^t = 1/|O| \sum_{v \in O} \mathbf{h}_v^{(k)}$.

Definition 6 service desk encoder When faced with multiple service desks available for the same medical activity, it is necessary to select an appropriate service desk for the patient to reduce waiting times. Upon determining the medical activity, the initial information of the problem provides the status information s_m for the service desk where the medical activity can be carried out. Since the service desk status information lacks a graph structure and each node represents the primary features of each service desk corresponding to each medical procedure, with no directed or undirected arcs between nodes, a fully connected layer is employed. A fully connected layer signifies that each neuron in the network is connected to all neurons in the previous layer. This structure enables the transmission of input service desk feature information to the next layer through weighted connections, facilitating global information interaction and impact. In this configuration, each neuron is connected to all neurons in the previous layer, allowing for a comprehensive consideration of the influence of the input service desk features and enhancing the model's learning capacity for complex relationships. In this study, a fully connected layer is utilized to encode the local states s_{mt} . At time t , the original local state

of the service desk s_{mt} is represented by $[H_t(M_{iq}), p_{j,i,q}]$. The service desk encoder outputs embedding vectors h_q^t for each service desk node and a pooling vector u^t .

Definition 7 decoders To enable two intelligent agents to execute actions a_{ot} and a_{mt} based on states s_{ot} and s_{mt} at time step t in scheduling, separate decoders are designed for each agent to process the outputs from the encoders. The decoders, composed of MLPs, share a similar network structure but do not have shared parameters. During the decoding process, each agent decodes the encoding independently using their respective decoder to generate scores for patient-medical activity selection $c_{t,v}^o$ and service desk selection $c_{t,q}^m$.

$$c_{t,v}^o = MLP_{\theta_{\pi_o}} \left(\left[h_{v,t}^{(K)}, h_G^t \right] \right), v \in 1, \dots, O \quad (16)$$

$$c_{t,q}^m = MLP_{\theta_{\pi_m}} \left(\left[h_t^q, u^t \right] \right), k \in 1, \dots, m \quad (17)$$

To ensure the legality of action selection for both intelligent agents, if a patient's medical activity has already been scheduled or violates priority constraints when scheduled, the corresponding score $c_{t,v}^o$ for the selected patient's medical activity and the score $c_{t,q}^m$ for the unavailable service desk are set to $-\infty$ to prevent the agents from selecting them. Subsequently, the Softmax function is applied to normalize $c_{t,v}^o$ and $c_{t,q}^m$, resulting in probability distributions $p_i(a_{ot})$ and $p_q(a_{mt})$ for actions a_{ot} and a_{mt} respectively. Finally, the patient-selecting agent employs greedy decision-making to select the action with the highest probability while the service desk agent selects actions based on ϵ , as shown in the following Eq. (18):

$$a_{mt} = \begin{cases} \arg \max_{a_{mt}^q} (p_q(a_{mt})), & \text{random} < \epsilon \\ \text{random.choice}(a_{mt}^q), & \text{random} > \epsilon \end{cases} \quad (18)$$

Deep learning methods effectively transform state information from graph structures into the decision-making processes of intelligent agents. Now, an algorithm is required to train these agents to make optimal decisions in the actual scheduling environment. In Sect. 4.3, this paper will introduce the PDQPN algorithm, an innovative approach that combines the strengths of PPO and DDQN. It not only enhances learning efficiency but also ensures that the scheduling strategies for patients developed in this study remain efficient and effective in the dynamically changing hospital environment.

4.3 Learning Algorithm

In the GTARS-DRL method, we introduce an innovative algorithm known as the Proximal Deep Q-Policy Network (PDQPN), specifically designed for the training and optimization of two critical agents in the outpatient scheduling problem: the patient-medical activity agent and the service desk agent. The PDQPN algorithm is engineered to enhance the decision-making capabilities of agents in both continuous and discrete action spaces by combining the strengths of Proximal Policy Optimization (PPO) and the Double Deep Q-Network (DDQN). The PPO component of the PDQPN algorithm is tailored for strategic learning by the patient-medical activity agent, capitalizing on historical experience to boost sample efficiency and optimize the expected rewards of patients navigating through medical processes. Concurrently, the DDQN element is deployed for the service desk agent's decision-making, employing two deep neural networks to refine the valuation of state-action pairs, thereby assisting patients in making optimal choices that minimize waiting times.

As elucidated in Fig. 5, the PDQPN algorithm fosters coordinated optimization of both agents' operations. Learning within the same environment, these agents interact based on their individual states and update their parameters in response to the rewards they receive. Moreover, the PDQPN leverages gradient descent to compute errors and refine network parameters, which enhances overall performance. The essence of the PDQPN algorithm is the separate training of the two agent types for distinct operations, with their learning processes being optimized using the respective algorithms of PPO and DDQN. This approach not only enables the learning of effective strategies to reduce patient wait times but also ensures adaptability to the dynamic and ever-changing hospital scheduling environment, facilitating real-time optimization. Table 5 shows the specific processes of PDQPN.

The PPO algorithm, as part of the PDQPN, is an enhanced policy gradient reinforcement learning algorithm. It introduces past experience data to enhance sample efficiency. To train a patient-medical activity-selecting intelligent agent,

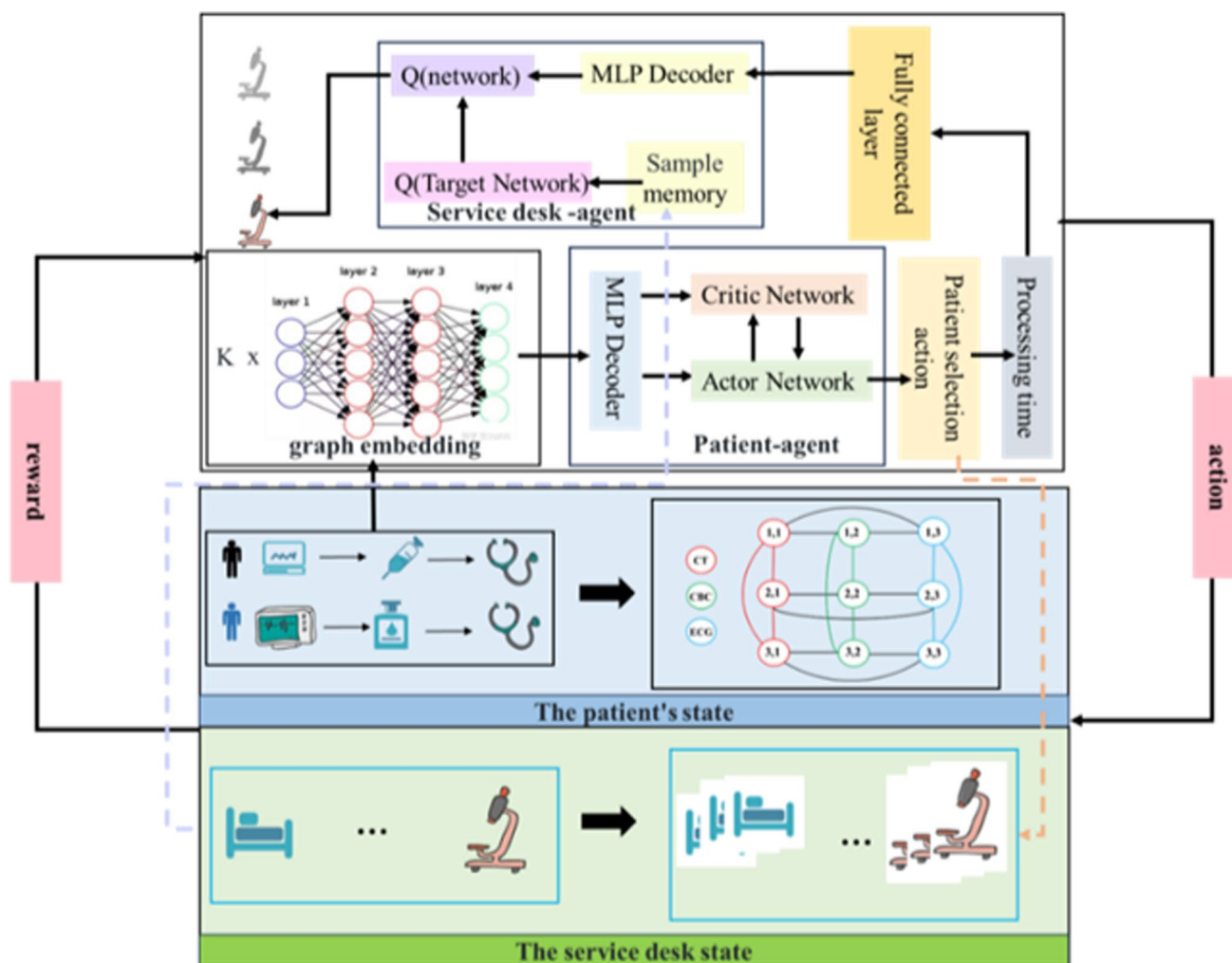


Fig. 5 The PDQPN algorithm

it is imperative to construct a neural network comprising two components: the policy network π_{θ} and the value network v_{ϕ} . The policy network generates the probability distribution of actions, forming the current decision-making policy, while the value network evaluates the value of the current state. By amalgamating the policy network and the value network, the PPO algorithm can train a more efficient and intelligent patient-selecting intelligent agent in the hospital patient scheduling dilemma.

The DDQN, which is part of the PDQPN framework, addresses the challenges of standard Q-Learning in handling continuous value functions. Specifically, in hospital scheduling and service desk selection, DDQN is utilized to circumvent overestimation issues that could lead to suboptimal choices being viewed as optimal. DDQN consists of two independent deep Q-networks: one network Q selects actions with maximum Q-value, while the other network Q' evaluates the value function $Q'(\theta)$ of state-action pairs. This approach allows for a more precise estimation of action value, eliminating the misleading effects of overestimation. By doing so, it enhances the training effectiveness and performance stability of the service desk selection agent within the PDQPN algorithm.

In the process of patient scheduling at the hospital, two intelligent agents continuously accumulate action experiences. Upon completion of the scheduling task, based on specific requirements, the patient-medical activity agent and the service desk selection intelligent agent will respectively utilize the PPO algorithm and DDQN algorithm to sample from the previous experience data and evaluate their performance. Subsequently, the model parameters of both agents are updated using the gradient descent algorithm to better adjust their decision-making strategies. Ultimately, the updated parameters result in the generation of a new policy, effectively enhancing the problem of hospital patient scheduling.

Table 5 GTARS-DRL algorithm flow pseudocode

Pseudocode for GMARS-DRL Algorithm
1: Initialize: Set training epochs E, agent update steps s, number of patients N, number of medical activities M, number of service desks Q, learning rate α, and initialize the replay buffer D. - For PPO algorithm: Initialize actor network π_{θ_o} with parameters θ_o, old actor network $\pi_{\theta_o}^{old}$ with parameters θ_o^{old}, critic network V_{ϕ} with parameters ϕ, clipping target parameter c_p, entropy coefficient c_e, clipping threshold ϵ, GAE parameter λ. - For DDQN algorithm: Initialize main network Q_{θ_m} with parameters θ_m, target network $Q_{\theta_m}^-$ with parameters θ_m^-, action selection probability parameter ϵ, update interval C. 3: For each episode $e = 1$ to E do 4: Sample B instances from the outpatient scheduling dataset. 5: For each batch $b = 1$ to B do 6: For each time step $t = 0$ to T do 7: Select action a_{bt}^o for patient-medical activity agent using policy π_{θ_o} based on the old actor network $\pi_{\theta_o}^{old}$ with greedy decision. 8: Select action a_{bt}^m for service desk agent using policy π_{θ_m} based on the old main network Q_{θ_m} with ϵ-greedy action selection. 9: Observe reward r_{bt} and next state $s_{b(t+1)}$ from the environment. 10: Calculate the GAE target using the rewards and state values: $\Delta_t = r_{bt} + \gamma * V(s_{b(t+1)}) - V(s_{bt})$ for all t. 11: Update the state value estimate $V(s_{bt})$ for the current state. 12: Calculate the probability ratio ζ for the action taken by the patient-medical activity agent: $\zeta_{b,t}(\theta_o) = \frac{\pi_{\theta_o}(a_{bt}^o s_{b,t})}{\pi_{\theta_o^{old}}(a_{bt}^o s_{b,t})}$ 13: Calculate the target y for the service desk agent using the main and target networks: $y = r_{bt} + \gamma * Q_{\theta_m}^-(a_{bt}^m, s_{b(t+1)}) \text{ if } t < T \text{ else } y = r_{bt}.$ 14: Perform gradient descent to update the parameters θ_o, ϕ, and θ_m using the actor loss L_{actor}, critic loss L_{critic}, and DDQN loss L_{DDQN}. 15: For each agent a in $[Agent1, Agent2]$ do 16: Calculate the L_{actor} for agent a: $L_{actor}(\theta_o) = - \sum_{b,t} \left[\min(r_t(\theta_o) A_{t-1}^{GAE}, \text{clip}(r_t(\theta_o), 1 - \epsilon, 1 + \epsilon) A_{t-1}^{GAE}) \right]$ 17: Calculate the critic's loss for agent a: $L_{critic}(\phi) = \sum_{b,t} \left[(V_{\phi}(s_{bt}) - \hat{V}_{bt})^2 \right]$ 18: If agent a is a service desk agent, calculate the DDQN loss: For each experience $(s_t, a_t, r_t + 1, s_{t+1})$ in the replay buffer do $L_{DDQN}(\theta_m) = E_{(s_t, a_t, r_{t+1}, s_{t+1}) \sim D} \left[\left(r_{t+1} + \gamma \cdot Q_{\theta_m}^-(s_{t+1}, \arg \max_{a'} Q(s_{t+1}, a'; \theta_m); \theta_m^-) - Q(s_t, a_t; \theta_m) \right)^2 \right]$ 19: For each agent a in $[Agent1, Agent2]$ do Calculate the gradients for L_{actor}, L_{critic} and L_{DDQN}. Update the policy network parameters θ_0: $\theta_0 = \theta_0 - \alpha \cdot \nabla_{\theta_0} L_{actor}(\theta_0)$ Update the critic network parameters ϕ: $\phi = \phi - \alpha \cdot \nabla_{\phi} L_{critic}(\phi)$ If agent a is a service desk agent, update the DDQN network parameters θ_m: $\theta_m = \theta_m - \alpha \cdot \nabla_{\theta_m} L_{DDQN}(\theta_m)$ 20: End time step loop for batch b. 21: Update the target network $Q_{\theta_m}^- = Q_{\theta_m}$ to stabilize the learning process in DDQN. 22: End batch loop. 23: End episode loop.

5 Numerical experiment

To validate the efficacy and superiority of the proposed deep reinforcement learning algorithm, this study employed simulation cases of varying scales for assessment. In constructing the cases, this study varied the number of patients and medical activities to create nine different scales of scenarios: 10×6 , 10×10 , 10×15 , 15×6 , 15×10 , 15×15 , 20×6 , 20×10 , and 20×15 . Within these nine scales, the number of service desks for each medical activity ranged from 1 to 3. Consequently, a total of 27 unique scenarios were generated, encompassing combinations of varying patient numbers, medical activities, and service desk quantities. Furthermore, to demonstrate the generalization capability of the proposed method, additional tests were conducted on larger-scale cases of 30×20 , 50×20 , and 100×20 .

5.1 Experiment design

In the experimental design, we first trained a patient medical activity selection agent using the PPO algorithm, with 10,000 iterations, 4 trajectory samples for policy network training, 100 validation instances, 2 GIN layer iterations ($K=2$), a hidden layer dimension of 64 for MLP, and hidden layer dimensions of 32 for both the action selection network and state value prediction network. The update network had 1 epoch, a clipping parameter ϵ of 0.2, a policy loss coefficient of 2, a value function coefficient of 1, an entropy coefficient of 0.01, a discount factor γ of 1, and a learning rate of 2×10^{-5} . Subsequently, based on the results of the previous step, we utilized the DDQN algorithm to train a service desk selection intelligent agent, setting the replay memory size to 2000, minibatch size to 32, linearly decreasing behavior policy from 1 to 0.1, a discount factor of 0.95, a learning rate of 0.00025, a target network update step of 100, and a replay start size of 100 [34, 38, 45]. This neural network was developed on Windows 10 64-bit using Python 3.7. The hardware specifications include an Intel(R) Core(TM) i9-9900 CPU and 64GB of RAM. The deep learning framework is built on Conda 11.7.0 and Torch 1.13.1.

Based on the states, actions, and rewards mentioned in Sect. 4, we constructed training environments for patient-medical activity agent and service desk selection agents. Experiments were carried out on instances with varying numbers of patients and medical activities to explore the impact of the number of service desks on the results of our algorithm. Figure 6a shows the results of outpatient scheduling problems that consider examinations only, while in Fig. 6b, the consultation activity is included. For Fig. 6c, all medical activities are incorporated in the experiment. Specifically, multiple service desks can be set up for the medical activities in the registration, examination, and medication retrieval phases. However, for the special phases of consultation and follow-up visits, since a specific doctor is chosen during the registration phase to diagnose the patient, the number of service desks for these two phases can only be one. It should be noted that in the experimental design of the number of service desks in this paper, only the registration, examination, and medication retrieval phases involve changes in the number of multiple service desks. The consultation and follow-up phases each have only one service desk.

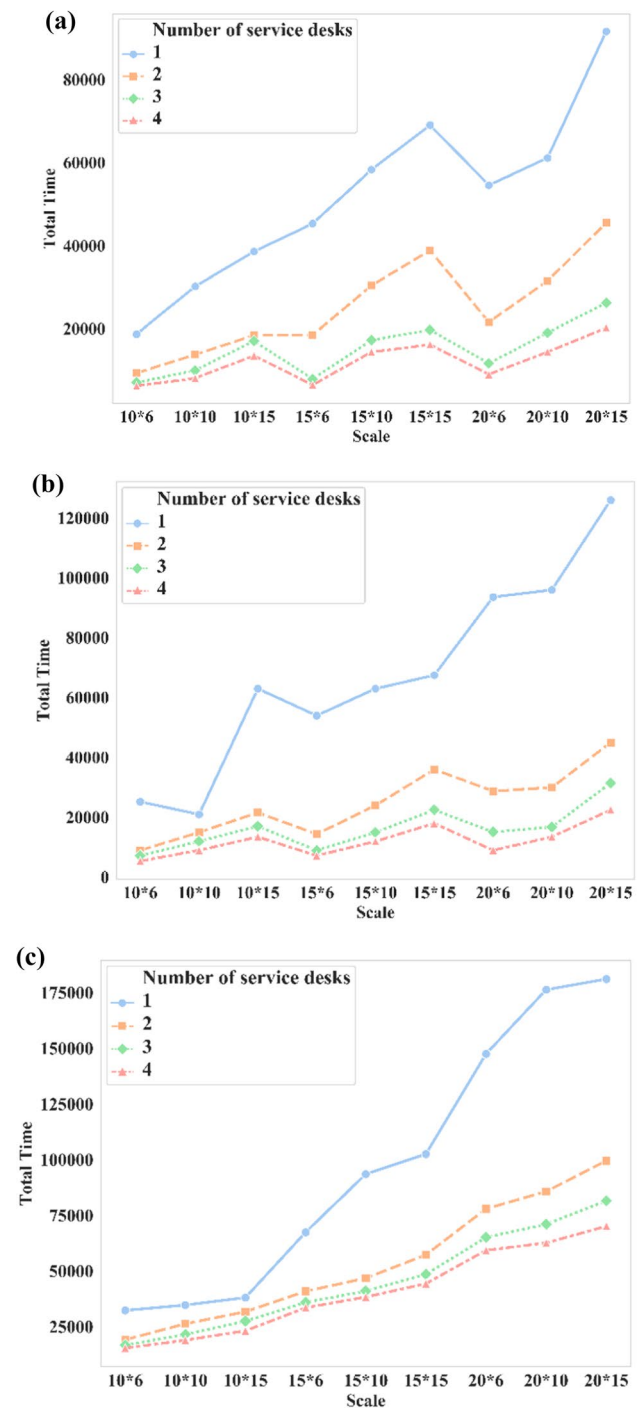
According to the results in Fig. 6, a notable change occurs with the increase in the number of service desks from one to two, leading to a significant decrease in the objective function value. Moreover, when the number of patients is within a small scale (i.e., less than or equal to 20), increasing the number of service desks to three or four does not significantly reduce the objective function further, yielding approximately similar results. Therefore, in the subsequent sections, the number of service desks is set as less than or equal to three.

Based on a 6×6 simulation instance, the curve of the total time for all patient medical activities after 10,000 rounds of training was plotted with the number of service desks varying from one to three, as depicted in Fig. 7. The horizontal axis represents the number of times a complete scheduling cycle is completed, while the vertical axis indicates the sum of all patient medical activity times for each training round. It is observed that as the number of training rounds increases, the total time for patient medical activities gradually decreases. With only one service desk, the training curve begins to converge around the 4,000 rounds. However, as the number of service desks increases, making the action space of the service desk selection agent more complex, the training curve with three service desks converges and stabilizes after 8000 rounds.

5.2 Algorithm performance

This study conducted a comparison of three traditional priority dispatching rules, the Shortest Processing Time (SPT) [46] rule, the Maximum Work Remaining (MWKR) [47] and the Most Operations Remaining (MOPNR) [48] rule.

Fig. 6 **a** Test results considering examinations only. **b** Test results considering examinations and consultation. **c** Test results including all medical activities



Among these, the SPT rule is the most widely utilized in the industrial sector. Additionally, this research compared the proposed algorithm with a genetic algorithm across various simulation scenarios that included different numbers of patients, medical activities, and service stations.

Table 6 presents the minimum total time for all patient medical activities obtained by each algorithm across the different scenarios. Initial testing was conducted on small-scale scenarios with dimensions of 10×6 , 10×10 , 10×15 , 15×6 , 15×10 , 15×15 , 20×6 , 20×10 and 20×15 . In these scenarios, the sequence of patient medical activities strictly follows the order of registration, consultation, examination, follow-up, and medication collection. Notably,

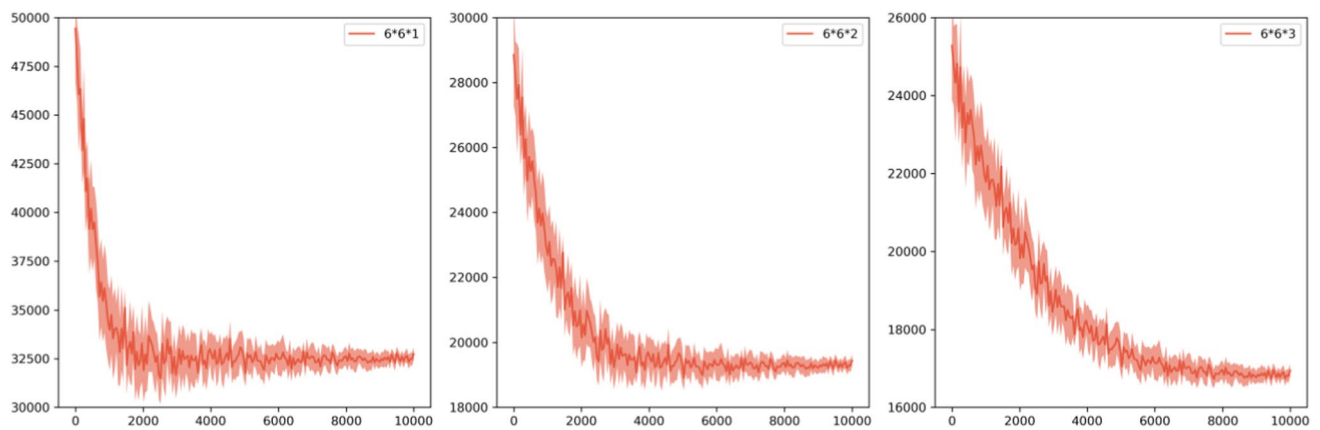


Fig. 7 Training results

Table 6 Comparison results

Number of patients	Number of medical activities	Number of service desks	SPT	MWKR	MOPNR	GA	PDQPN
10	6	1	49,347	46,847	44,991	32,076	32,456
		2	26,772	25,786	24,952	14,830	19,278
		3	20,998	22,292	19,992	16,075	16,886
	10	1	47,679	50,446	45,890	34,303	34,898
		2	39,051	38,043	36,998	25,218	26,469
		3	30,704	29,572	27,525	20,986	21,654
	15	1	59,447	62,760	65,158	28,558	38,160
		2	54,281	51,681	49,820	33,863	31,857
		3	39,620	38,269	37,016	25,719	27,648
15	6	1	98,072	92,188	87,577	66,689	67,521
		2	49,763	48,171	46,773	39,046	40,980
		3	47,545	45,687	44,179	34,989	36,036
	10	1	130,000	126,853	106,744	90,268	93,678
		2	74,562	61,604	55,344	45,538	46,860
		3	56,378	64,150	55,645	41,100	41,100
	15	1	140,351	131,087	128,203	100,463	102,600
		2	74,296	70,343	64,190	52,659	57,360
		3	75,269	73,252	65,430	47,825	48,684
20	6	1	233,548	230,509	209,812	148,603	147,600
		2	176,370	137,273	133,936	72,929	78,039
		3	109,766	100,438	90,305	58,471	65,200
	10	1	300,045	282,747	273,537	183,626	181,258
		2	118,027	151,615	123,458	84,859	85,831
		3	94,252	91,801	87,402	65,963	71,145
	15	1	312,426	304,093	284,803	191,742	176,419
		2	162,830	158,676	145,733	94,242	99,626
		3	116,172	114,370	110,163	78,356	81,678

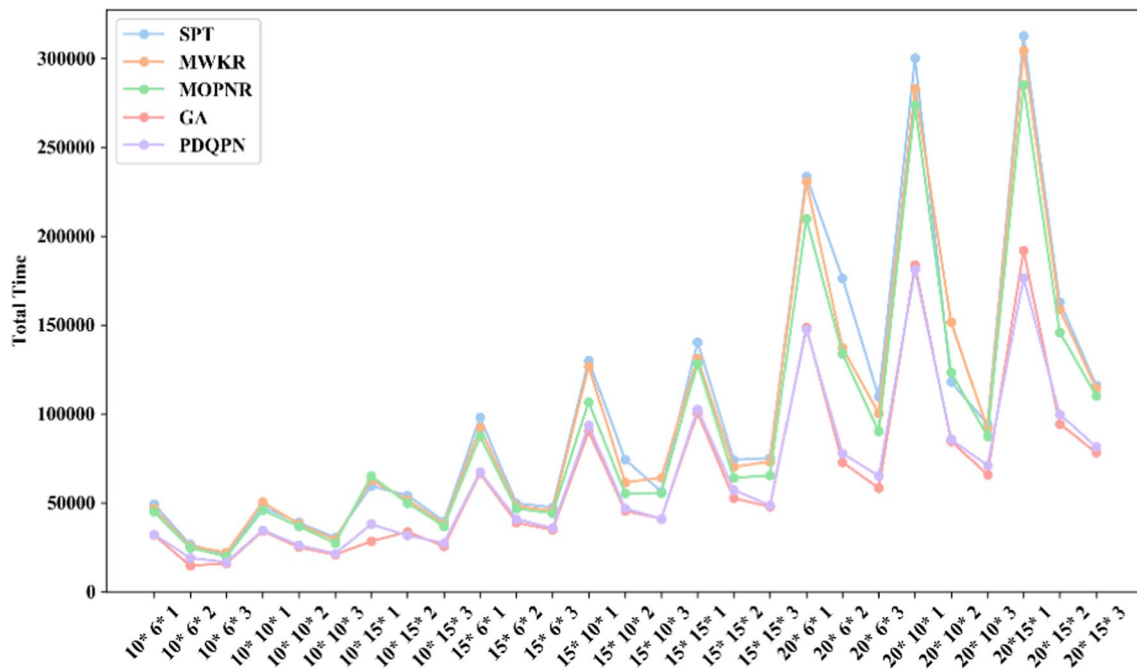


Fig. 8 Comparison results of various algorithms

during the examination phase, the sequence of specific medical procedures required by patients is determined by a patient medical activity selection intelligent agent. Importantly, to more closely mirror real-world conditions, the test scenarios were designed such that only one service desk is available for both consultation and follow-up activities, while the number of service desks for other medical activities was adjusted for testing purposes.

Based on Table 5 and Fig. 8, PDQPN consistently outperforms traditional scheduling rules in solution outcomes. While traditional rules perform adequately in small scenarios like 10×6 , as the scale increases to 20×6 , 20×10 , and 20×15 , PDQPN's superiority becomes evident. The disparity between PDQPN and the optimal solution remains stable, unlike traditional rules whose performance disparity grows with instance scale. When using a single service desk, PDQPN and GA results are comparable, but PDQPN consistently outperforms GA when the number of service desks increases. Overall, while both deep learning and genetic algorithms show close results, PDQPN demonstrates superior performance in most cases, with only a few instances favoring genetic algorithms.

It is crucial to note that the computational time for genetic algorithms significantly exceeds that of the deep reinforcement learning method. Therefore, to further compare the performance of traditional scheduling rules, metaheuristic algorithms, and DRL algorithms, we conducted a comparison of the Relative Percentage Difference (RPD) among the results obtained by various algorithms. RPD refers to the relative percentage difference between the optimal values obtained by different algorithms and the actual optimal solution, calculated as shown in Eq. (5-1): where C represents the result obtained by the corresponding algorithm, and C^* denotes the optimal solution for the case.

$$RPD = \frac{C - C^*}{C^*} \times 100 \quad (19)$$

From Fig. 9, it can be concluded that the average RPD of PDQPN is significantly lower than that of SPT, MWKR, and MOPNR, with a relatively shorter distance between the upper and lower limits of the box plot, indicating a smaller relative deviation. Furthermore, the median RPD of PDQPN is only slightly above that of the GA algorithm, with a small difference between the two. To provide a detailed comparison of the computational efficiency of the GA and DRL algorithms, we further calculated the following evaluation metrics: Average Relative Percentage Deviation (ARPD), Average CPU Time (ACPU), and Average Relative Percentage Time (ARPT) [49]. The ARPD, ACPU, and ARPT for algorithm p are defined as follows:

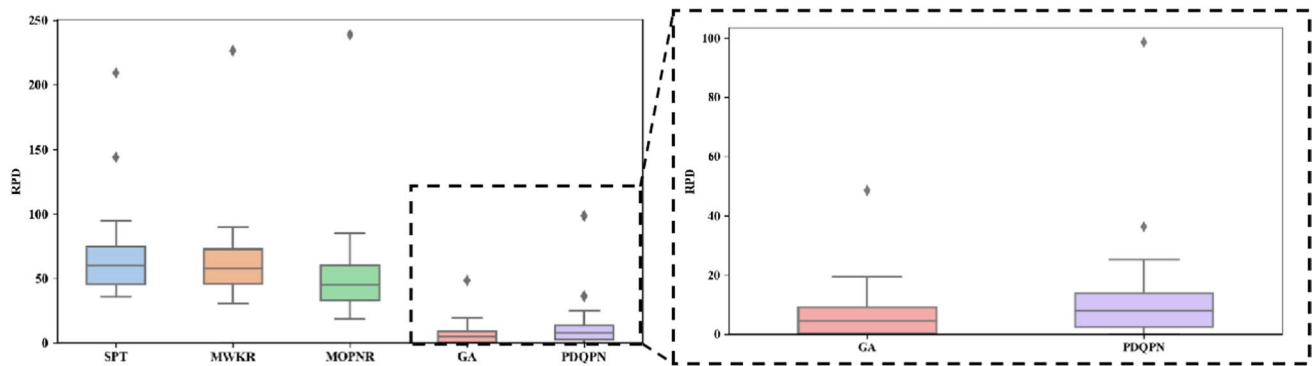


Fig. 9 The RPD values of various algorithms

$$ARPD_p = \frac{\sum_{j=1}^J RPD_{j;p}}{J}, ACPU_p = \frac{\sum_{j=1}^J CPU_{j;p}}{J}, ARPT_p = \frac{\sum_{j=1}^J \left(\frac{CPU_{j;p} - ACT_j}{ACT_j} \right)}{J} + 1,$$

where $ARPD_p$ represents the computation time of algorithm p on instance j , J the total number of instances, and P the total number of algorithms considered for comparison in the experiment. ACPU and ARPT are two performance indicators used to measure computational efficiency, while ARPD is used to assess the quality of the solutions. The ARPD, ACPU, and ARPT for each algorithm running on simulation instances of size 10×6 , 10×10 , 10×15 , 15×6 , 15×10 , 15×15 , 20×6 , 20×10 and 20×15 are presented in Table 7.

Given Table 7 and Fig. 10, it is evident that although the GA achieves the best solution quality, its computational efficiency is the poorest. In comparison with the other three algorithms, only the average computation time of the GA is not within the same order of magnitude as the others. This is due to the limitations of metaheuristic algorithms, which require resetting and recalculating the algorithm when the problem changes, and lack generalization ability, resulting in longer computation times. Overall, PDQPN can achieve a high level of solution quality associated with a solution time close to scheduling rules. Therefore, the algorithm proposed in this paper has certain advantages in solving hospital patient scheduling problems.

The performance of the proposed solving algorithm is evaluated through large-scale simulation cases. Models trained on simulation cases with sizes 15×15 , 20×10 , and 20×15 are utilized to solve large-scale instances with sizes 30×20 , 50×20 , and 100×20 , respectively. The experimental results, as summarized in Table 8, demonstrate that the models trained on small-scale instances perform well on these larger instances, providing solutions significantly superior to

Table 7 Summary of ARPD, ACPU and ARPT of algorithms

Algorithm	ARPD	ACPU	ARPT
SPT	68.27	0.229	0.078
MWKR	63.88	0.238	0.083
MOPNR	53.73	0.23	0.081
GA	6.78	14.525	4.430
PDQPN	12.57	0.762	0.327

Fig. 10 Computation efficiency of various algorithms

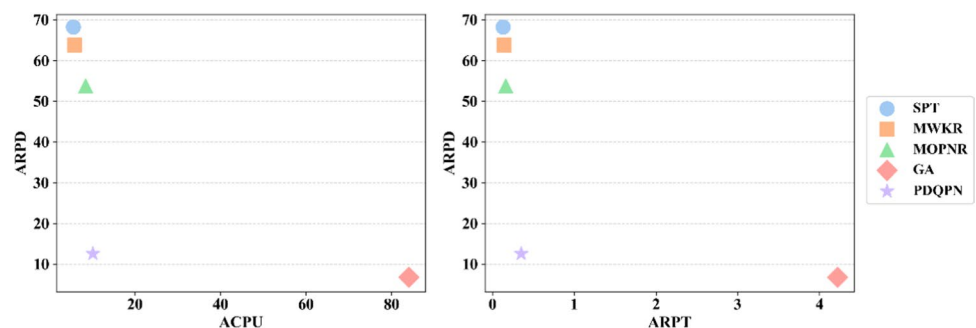
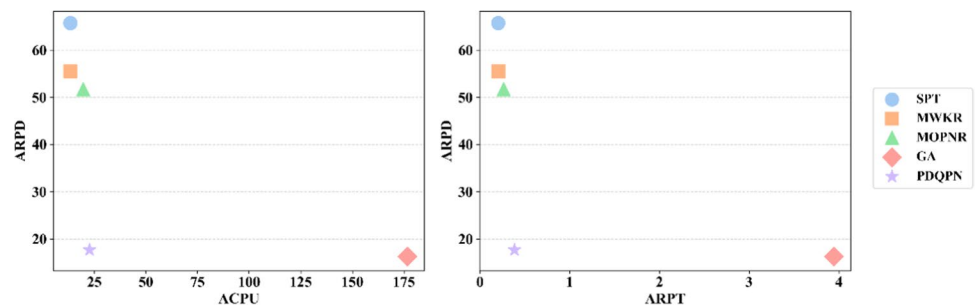


Table 8 The results are compared under large-scale examples

Number of patients	Number of medical activities	Number of service desks	SPT	MWKR	MOPNR	GA	PDQPN
30	20	1	830,356	738,620	790,497	504,523	514,820
		2	403,750	376,335	375,926	291,429	290,700
		3	380,869	358,817	348,002	261,799	262,805
50	20	1	1,965,827	1,853,243	1,707,578	1,346,887	1,362,032
		2	1,201,249	1,128,080	1,145,030	892,768	892,501
		3	937,923	898,908	834,842	676,803	686,970
100	20	1	7,035,403	6,824,341	6,227,211	4,869,203	5,041,627
		2	4,331,526	4,201,850	3,949,480	3,162,205	3,162,014
		3	3,338,831	3,038,336	3,099,103	2,547,512	2,633,640

Fig. 11 The computational efficiency of various algorithms under large-scale examples**Table 9** ARPD, ACPU and ARPT of each algorithm under the large-scale example

Algorithm	ARPD	ACPU	ARPT
SPT	65.759	13.345	0.203
MWKR	55.513	13.422	0.205
MOPNR	51.720	19.530	0.263
GA	16.322	176.633	3.943
PDQPN	17.773	22.653	0.385

traditional PDR approaches. This observation suggests the practical utility of the proposed method in effectively extracting knowledge from small-scale simulation cases for solving large-scale problems.

To evaluate the computational efficiency and quality of the proposed solution for large-scale scheduling problems, the ARPD, ACPU and ARPT indicators are calculated for the results obtained from running large-scale instances. Figure 11 and Table 9 demonstrate that PDQPN exhibits high computational efficiency, providing high-quality solutions for large-scale instances within an average of 30 s, while achieving an ARPD comparable to that of GA. Therefore, the proposed algorithm not only exhibits superiority but also meets the demands of real-world applications.

6 Conclusion

The GTARS-DRL algorithm proposed in this study offers hospitals a novel tool to address the growing challenge of balancing escalating healthcare service demands with limited resources. By implementing the GTARS-DRL in hospitals of varying scales, significant reductions in patient wait times can be achieved, directly impacting the operational efficiency and overall patient experience of the hospital. Moreover, the optimization of resource allocation enables hospitals to better manage fluctuations in patient flow and enhances their responsiveness during emergencies, thereby improving the quality and accessibility of medical services overall. Specifically, in small community hospitals, the GTARS-DRL algorithm

can assist in the more effective distribution of scarce medical resources, while in large medical centers, it can enhance the management efficiency of complex systems by optimizing patient processes.

To address the increasing conflict between the growing medical needs and limited medical resources, this paper proposes a solution for outpatient scheduling to minimize patient waiting time, enhance patient satisfaction, improve the efficient utilization of hospital medical resources, and promote the development of smart hospitals. The contributions of this paper are as follows:

- i. The MDP formulation for outpatient scheduling is proposed, with the states, actions, state transitions, and rewards in the outpatient scheduling environment described. Specifically, a disjunctive graph is utilized to represent the local states of the physician–patient activity relationship.
- ii. To tackle the MDP problem, the GTARS-DRL method is designed and developed to maximize the reduction of the total time required for patients to complete all medical activities in the hospital, including determining the sequence of medical activities, the order of different patients, and the selection of service desks.
- iii. The PDQPN algorithm is proposed to combine both value and policy based reinforcement learning methods for achieving collaborative training of two agents in GTARS-DRL, thereby optimizing the quality of solutions to outpatient scheduling problems.
- iv. A specific mathematical model is established for outpatient scheduling issues.

The conclusions of this paper are as follows:

- i. The GTARS-DRL method effectively optimizes outpatient patient scheduling, reducing patient waiting time in the hospital.
- ii. Experimental results demonstrate that the PDQPN method significantly outperforms traditional scheduling rules in solving outpatient scheduling problems.
- iii. The solution quality of PDQPN is comparable to genetic algorithms (GA), yet its efficiency far exceeds that of GA, demonstrating the superiority of PDQPN.

This study is directed towards smart hospitals, which typically employ electronic health record systems capable of automatically collecting and storing patient information, medical activities, treatment outcomes, and patient feedback. The digitization of records reduces errors and omissions associated with paper records, enhancing the accuracy and accessibility of data. Smart hospitals leverage Internet of Things (IoT) technology for real-time monitoring of patient medical activity status and the condition of medical equipment. Moreover, the information systems in smart hospitals are usually integrated, allowing different medical departments and devices to share data seamlessly. This integration reduces data silos and improves data consistency and integrity. Such high-quality, real-time data is particularly crucial for intelligent patient scheduling. However, for developing countries, obtaining this type of data may pose certain challenges. To address this issue, future work could further develop data preprocessing and cleansing techniques to enhance data quality and mitigate the impact of noise and outliers on algorithm performance. Additionally, exploring the use of proxy indicators and simulated data to enrich the learning process could enable effective scheduling in environments where data is limited.

To apply the GTARS-DRL algorithm in a real-world smart hospital environment, several key implementation steps need to be considered. Hospitals must contemplate the integration of the algorithm with existing information systems, including patient registration, appointment scheduling, and resource management systems. Furthermore, hospitals may need to invest in necessary hardware, such as high-performance servers, and software, such as programming environments for algorithm implementation and simulation. Secondly, hospital staff must be trained to understand the principles of the algorithm and be capable of monitoring and adjusting its outputs. Lastly, hospitals should establish feedback mechanisms to gather feedback from patients and medical staff on the scheduling system, allowing for continuous optimization of the algorithm's performance. Considering the uncertainty of patient behavior and medical demands, hospitals should also develop contingency plans to address the algorithm's needs for adjustment in the face of exceptional circumstances.

The GTARS-DRL algorithm, while demonstrating excellent performance in simulated environments, requires future research to address its limitations, particularly in managing the uncertainties and dynamics of real-world hospital operations. The algorithm currently assumes punctual patient arrivals and consistent urgency levels, which may not always

be applicable. Future work will thus extend the algorithm to account for a broader range of patient behaviors, including no-shows, variable arrival times, and walk-in patients. It will also consider diverse arrival patterns, seasonal variations, and simulate patient flow surges due to atypical events, providing a more realistic portrayal of real-world uncertainties. Moreover, the impact of varying medical staff levels on the algorithm's performance will be evaluated, ensuring efficient scheduling even amidst staff shortages or surpluses. Critically, future studies will develop scenarios to mimic unpredictable disruptions like equipment failures or emergencies, thereby enhancing the algorithm's adaptability and reliability.

Although the current study concentrates on outpatient scheduling in smart hospitals, the GTARS-DRL model is envisioned to be applicable to other medical scheduling domains. The inherent complexity of outpatient scheduling offers a solid foundation for developing algorithms that are both flexible and efficient. Future research will explore the GTARS-DRL algorithm's application in various scenarios, including inpatient and surgical scheduling, which present unique challenges in resource allocation, patient flow, and priority rules. The algorithm will be adapted to meet the specific needs of these scenarios, considering factors such as the urgency of surgeries and the continuity of inpatient care. Collaboration with healthcare professionals will be sought to understand the unique challenges in these areas and ensure the algorithm meets clinical demands. The algorithm's multi-objective optimization capabilities will be enhanced to balance time efficiency with the quality of patient care, extending its utility beyond outpatient settings to a broader spectrum of healthcare applications.

To overcome the limitations identified in this paper, future research will integrate advanced predictive models for more precise forecasting of patient arrivals and staff availability, thus reducing reliance on static patterns. The development of more agile scheduling strategies will enable the algorithm to swiftly adapt to real-time shifts in resources and demands. Furthermore, the research will examine the algorithm's suitability for hospitals of diverse sizes and types and explore customization options based on regional medical demands and resource limitations. By incorporating these enhancements, the GTARS-DRL algorithm will be better equipped to navigate the complexities of real-world healthcare systems, significantly amplifying its applicability on a global scale. This advancement is expected to not only propel forward the field of hospital outpatient scheduling but also yield tangible benefits for hospital management and patient care, even in the face of large-scale public health emergencies or equipment failures during medical activities.

Author contributions Ms. Jianing Zuo conceptualized and designed the algorithm, implemented the initial codebase, and prepared the original manuscript draft. Prof Yan Jin provided essential theoretical insights, contributed to algorithm improvements, and critically revised the manuscript for important intellectual content. Prof Weibo Liu supervised the project, provided strategic direction in algorithm development and testing, and conducted a thorough review and final approval of the manuscript prior to submission.

Funding The study was funded by Youth Foundation of Social Science and Humanity, China Ministry of Education (21YJC630084); Youth Foundation of Shandong Natural Science (ZR2021QG001); National Natural Science Foundation of China (72101136).

Data availability The data that support the findings of this study are available on request from the corresponding author.

Declarations

Competing interests The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

References

1. Canellas MM, Pachamanova DA, Perakis G, Lami OS, Tsiourvas A. A granular approach to optimal and fair patient placement in hospital emergency departments. *Prod Oper Manag.* 2024. <https://doi.org/10.1177/10591478241240390>.
2. Wang Q, Carson AL, Zheng S. EXPRESS: the relative indirect effects of technology bias and implicit bias on racial disparity in service delivery and sepsis mortality. *Prod Oper Manag.* 2024. <https://doi.org/10.1177/10591478241270115>.

3. Babashov V, Sauré A, Ozturk O, Patrick J. Setting wait time targets in a multi-priority patient setting. *Prod Oper Manag.* 2023;32(6):1958–74. <https://doi.org/10.1111/poms.13951>.
4. Reihaneh M, Ansari S, Farhadi F. Patient appointment scheduling at hemodialysis centers: an exact branch and price approach. *Eur J Oper Res.* 2023;309(1):35–52. <https://doi.org/10.1016/j.ejor.2023.01.024>.
5. Shao K, Fan W, Lan S, Kong M, Yang S. A column generation-based heuristic for brachytherapy patient scheduling with multiple treatment sessions considering radioactive source decay and time constraints. *Omega.* 2023;118: 102853. <https://doi.org/10.1016/j.omega.2023.102853>.
6. Alves de Queiroz T, Iori M, Kramer A, Kuo YH. Dynamic scheduling of patients in emergency departments. *Eur J Oper Res.* 2023;310(1):100–16. <https://doi.org/10.1016/j.ejor.2023.03.004>.
7. Kasapidis GA, Paraskevopoulos DC, Repoussis PP, Tarantilis CD. Flexible job shop scheduling problems with arbitrary precedence graphs. *Prod Oper Manag.* 2021;30(11):4044–68. <https://doi.org/10.1111/poms.13501>.
8. Golmohammadi D, Zhao L, Dreyfus D. Using machine learning techniques to reduce uncertainty for outpatient appointment scheduling practices in outpatient clinics. *Omega.* 2023;120: 102907. <https://doi.org/10.1016/j.omega.2023.102907>.
9. Kuiper A, Mandjes M. Appointment scheduling in tandem-type service systems. *Omega.* 2015;57:145–56. <https://doi.org/10.1016/j.omega.2015.04.009>.
10. Soltani M, Samorani M, Kolfal B. Appointment scheduling with multiple providers and stochastic service times. *Eur J Oper Res.* 2019;277(2):667–83. <https://doi.org/10.1016/j.ejor.2019.02.051>.
11. Schoenfelder J, Bretthauer KM, Wright PD, Coe E. Nurse scheduling with quick-response methods: improving hospital performance, nurse workload, and patient experience. *Eur J Oper Res.* 2020;283(1):390–403. <https://doi.org/10.1016/j.ejor.2019.10.047>.
12. Hesarakı AF, Dellaert NP, de Kok T. Generating outpatient chemotherapy appointment templates with balanced flowtime and makespan. *Eur J Oper Res.* 2019;275(1):304–18. <https://doi.org/10.1016/j.ejor.2018.11.028>.
13. Wang K, Qin H, Huang Y, Luo M, Zhou L. Surgery scheduling in outpatient procedure centre with re-entrant patient flow and fuzzy service times. *Omega.* 2021;102: 102350. <https://doi.org/10.1016/j.omega.2020.102350>.
14. Zhang J, Dridi M, El Moudni A. Column-generation-based heuristic approaches to stochastic surgery scheduling with downstream capacity constraints. *Int J Prod Econ.* 2020;229: 107764. <https://doi.org/10.1016/j.ijpe.2020.107764>.
15. Đurasević M, Jakobović D. A survey of dispatching rules for the dynamic unrelated machines environment. *Expert Syst Appl.* 2018;113:555–69. <https://doi.org/10.1016/j.eswa.2018.06.053>.
16. Deliktaş D, Özcan E, Ustun O, Torkul O. Evolutionary algorithms for multi-objective flexible job shop cell scheduling. *Appl Soft Comput.* 2021;113: 107890. <https://doi.org/10.1016/j.asoc.2021.107890>.
17. Xin P, Sun T, Wang J, Zhang N, Li Y. A review of production scheduling research based on genetic algorithm. *Lecture notes on data engineering and communications technologies*; 2023. https://doi.org/10.1007/978-3-031-29097-8_52.
18. Chen N, Xie N, Wang Y. An elite genetic algorithm for flexible job shop scheduling problem with extracted grey processing time. *Appl Soft Comput.* 2022;131: 109783. <https://doi.org/10.1016/j.asoc.2022.109783>.
19. Missaoui, A., & Boujelbene, Y. Hybrid Genetic algorithm for Blocking Flowshop scheduling with due date windows. In: 2021 international conference on decision aid sciences and application (DASA); 2021. <https://doi.org/10.1109/dasa53625.2021.9682275>.
20. Liu Z, Wang J, Zhang C, Chu H, Ding G, Zhang L. A hybrid genetic-particle swarm algorithm based on multilevel neighbourhood structure for flexible job shop scheduling problem. *Comput Oper Res.* 2021;135: 105431. <https://doi.org/10.1016/j.cor.2021.105431>.
21. Marichelvam M, Geetha M, Tosun M. An improved particle swarm optimization algorithm to solve hybrid flowshop scheduling problems with the effect of human factors—a case study. *Comput Oper Res.* 2020;114: 104812. <https://doi.org/10.1016/j.cor.2019.104812>.
22. Zhao B, Gao J, Chen K, Guo K. Two-generation Pareto ant colony algorithm for multi-objective job shop scheduling problem with alternative process plans and unrelated parallel machines. *J Intell Manuf.* 2015;29(1):93–108. <https://doi.org/10.1007/s10845-015-1091-z>.
23. Jieran Y, Aimin W, Yan G, Xinyi S. An improved grey wolf optimizer for flexible job-shop scheduling problem. 2020 IEEE 11th international conference on mechanical and intelligent manufacturing technologies (ICMIMT); 2020. <https://doi.org/10.1109/icmimt49010.2020.9041184>.
24. Li Y, Huang W, Wu R, Guo K. An improved artificial bee colony algorithm for solving multi-objective low-carbon flexible job shop scheduling problem. *Appl Soft Comput.* 2020;95: 106544. <https://doi.org/10.1016/j.asoc.2020.106544>.
25. Yang W, Su J, Yao Y, Yang Z, Yuan Y. A novel hybrid whale optimization algorithm for flexible job-shop scheduling problem. *Machines.* 2022;10(8):618. <https://doi.org/10.3390/machines10080618>.
26. Li X, Xie J, Ma Q, Gao L, Li P. Improved gray wolf optimizer for distributed flexible job shop scheduling problem. *Sci China Technol Sci.* 2022;65(9):2105–15. <https://doi.org/10.1007/s11431-022-2096-6>.
27. Shivasankaran N, Kumar PS, Raja KV. Hybrid sorting immune simulated annealing algorithm for flexible job shop scheduling. *Int J Comput Intell Syst.* 2015;8(3):455. <https://doi.org/10.1080/18756891.2015.1017383>.
28. Cao Y, Shi H, Chang D. Differential evolution algorithm with dynamic multi-population applied to flexible job shop schedule. *Eng Opt.* 2021;54(3):387–408. <https://doi.org/10.1080/0305215x.2021.1872067>.
29. Sun J, Zhang G, Lu J, Zhang W. A hybrid many-objective evolutionary algorithm for flexible job-shop scheduling problem with transportation and setup times. *Comput Oper Res.* 2021;132: 105263. <https://doi.org/10.1016/j.cor.2021.105263>.
30. Bouazza W, Salles Y, Beldjilali B. A distributed approach solving partially flexible job-shop scheduling problem with a Q-learning effect. *IFAC-PapersOnLine.* 2017;50(1):15890–5. <https://doi.org/10.1016/j.ifacol.2017.08.2354>.
31. Luo S. Dynamic scheduling for flexible job shop with new job insertions by deep reinforcement learning. *Appl Soft Comput.* 2020;91: 106208. <https://doi.org/10.1016/j.asoc.2020.106208>.
32. Xue T, Zeng P, Yu H. A reinforcement learning method for multi-AGV scheduling in manufacturing. In: 2018 IEEE International Conference on Industrial Technology (ICIT); 2018. <https://doi.org/10.1109/icit.2018.8352413>.
33. Liu R, Piplani R, Toro C. Deep reinforcement learning for dynamic scheduling of a flexible job shop. *Int J Prod Res.* 2022;60(13):4049–69. <https://doi.org/10.1080/00207543.2022.2058432>.

34. Chang J, Yu D, Hu Y, He W, Yu H. Deep reinforcement learning for dynamic flexible job shop scheduling with random job arrival. *Processes*. 2022;10(4):760. <https://doi.org/10.3390/pr10040760>.
35. Luo S, Zhang L, Fan Y. Real-time scheduling for dynamic partial-no-wait multiobjective flexible job shop by deep reinforcement learning. *IEEE Trans Autom Sci Eng*. 2022;19(4):3020–38. <https://doi.org/10.1109/tase.2021.3104716>.
36. Gil CB, Lee JH. Deep reinforcement learning approach for material scheduling considering high-dimensional environment of hybrid flow-shop problem. *Appl Sci*. 2022;12(18):9332. <https://doi.org/10.3390/app12189332>.
37. Gui Y, Tang D, Zhu H, Zhang Y, Zhang Z. Dynamic scheduling for flexible job shop using a deep reinforcement learning approach. *Comput Ind Eng*. 2023;180: 109255. <https://doi.org/10.1016/j.cie.2023.109255>.
38. Zhang C, Song W, Cao Z, Zhang J, Xu C. Learning to dispatch for job shop scheduling via deep reinforcement learning; 2020. <https://doi.org/10.48550/arXiv.2010.12367>.
39. Park J, Chun J, Kim SH, Kim Y, Park J. Learning to schedule job-shop problems: representation and policy learning using graph neural network and reinforcement learning. *Int J Prod Res*. 2021;59(11):3360–77. <https://doi.org/10.1080/00207543.2020.1870013>.
40. Han BA, Yang JJ. A deep reinforcement learning based solution for flexible job shop scheduling problem. *Int J Simul Model*. 2021;20(2):375–86. <https://doi.org/10.2507/ijsimm20-2-co7>.
41. Ren J, Ye C, Yang F. Solving flow-shop scheduling problem with a reinforcement learning algorithm that generalizes the value function with neural network. *Alexandria Eng J*. 2021;60(3):2787–800. <https://doi.org/10.1016/j.aej.2021.01.030>.
42. Song W, Chen X, Li Q, Cao Z. Flexible job-shop scheduling via graph neural network and deep reinforcement learning. *IEEE Trans Ind Inf*. 2023;19(2):1600–10. <https://doi.org/10.1109/tii.2022.3189725>.
43. Jing X, Yao X, Liu M, Zhou J. Two-agent reinforcement learning based on graph convolutional network for flexible job shop scheduling. *J Intell Manuf*. 2022;35(1):75–93. <https://doi.org/10.1007/s10845-022-02037-5>.
44. Lei K, Guo P, Zhao W, Wang Y, Qian L, Meng X, Tang L. A multi-action deep reinforcement learning framework for flexible Job-shop scheduling problem. *Expert Syst Appl*. 2022;205: 117796. <https://doi.org/10.1016/j.eswa.2022.117796>.
45. Nair A, Srinivasan P, Blackwell S, Alcicek C, Fearon R, De Maria A, et al. Massively parallel methods for deep reinforcement learning. *Comput Sci*. 2015. <https://doi.org/10.48550/arXiv.1507.04296>.
46. Grabot B, Geneste L. Dispatching rules in scheduling dispatching rules in scheduling: a fuzzy approach. *Int J Prod Res*. 1994;32(4):903–15. <https://doi.org/10.1080/00207549408956978>.
47. Sels V, Gheysen N, Vanhoucke M. A comparison of priority rules for the job shop scheduling problem under different flow time- and tardiness-related objective functions. *Int J Prod Res*. 2012;50(15):4255–70. <https://doi.org/10.1080/00207543.2011.611539>.
48. Panwalkar SS, Iskander W. A survey of scheduling rules. *Oper Res*. 1977;25(1):45–61. <https://doi.org/10.1287/opre.25.1.45>.
49. Fernandez-Viagas V, Ruiz R, Framinan JM. A new vision of approximate methods for the permutation flowshop to minimise makespan: State-of-the-art and computational evaluation. *Eur J Oper Res*. 2017;257(3):707–21. <https://doi.org/10.1016/j.ejor.2016.09.055>.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.